

Civic Operations

Explainable City Service Triage and Field Crew Dispatch

Group Project Report

Practice Module, Graduate Certificate in Intelligent Reasoning Systems

NUS Institute of Systems Science (ISS), Master of Technology in Artificial Intelligence Systems

Team MANTIS (Group 23) · Batch AIS08FT · Submitted 23rd October 2026

MANTIS: Machine Intelligence, Analytics, Networks, Technology and Intelligent Systems

Member	Student ID	Components owned
Khalid Alhazmi	A0361128J	Urgency, Prioritisation
Mohammad Alharthi	A0354097W	Agency and skill routing, External factors, Crew deployment and routing
Nabeel Taresh	A0352296X	Duplicate detection, Fake request detection, Admin questions
Indish Sankalp Sandeep	A0351383E	Intake bot, Emergency check, Credibility and escalation, Predictive alerting, application and integration

Acknowledgement

We thank the teaching and non-teaching team of the Intelligent Reasoning Systems practice module at NUS ISS for their guidance, and for the questions at our proposal presentation that sharpened the design. We thank NYC Open Data and the OpenStreetMap contributors, whose open data made a realistic system possible, and the authors of the open source libraries in section 3.1. AI assistance was used for drafting, formatting and code review. Every design decision, every component and every result belongs to the team, and the team has checked each one.

Executive summary

At eight in the morning a resident types one line: there is a deep pothole on Atlantic Avenue, a cyclist nearly fell. Within seconds that line has been checked for danger, read, sent to the Department of Transportation, compared with every open report nearby, matched to a childcare centre a few steps away, ranked against 140 other requests and put on a crew's route along real Brooklyn streets. Every one of those steps can be explained.

New York City's 311 service receives millions of non-emergency requests a year [1]. Each one has to be read, sent to the right agency, ranked against everything else that is open, and given to a crew. Today much of that triage is manual, and the reasons behind it are invisible to the resident who reported the problem.

Civic Operations is a working system that does this triage with intelligent reasoning and explains every step. Twelve AI components read a report, stop anything that belongs with 911, route it to an agency, link duplicates, notice schools and hospitals nearby, weigh the reporter's track record and the risk that a report is unfounded, assign an urgency, rank it against every open request with the city's own service deadlines, and plan crew routes on the real road network. Routes go to the crews automatically, and city staff can pull any job back or give it to another crew. Crews record what they found, and that outcome flows back to the reporter's record and to the resident.

It runs as four web portals over one API in two Docker containers, on 150 real NYC 311 requests. Each component sits behind an adapter with a safe default, so a failure never stops the service, and the tech team can switch any component off and measure the effect. On 34 hand written scenarios, switching off Prioritisation component changes the queue order to a Kendall tau b of 0.364, Duplicate detection to 0.630 and External factors to 0.982. The crew planner's genetic algorithm serves 9 to 18% more requests than dispatching strictly in queue order, with 38 to 48% less travel. Across the project the team compared 47 methods in 1,679 experiment runs, and section 4 reports every one.

Try it live: http://157.230.191.175/ (plain HTTP, see section 3.6, and it may be taken down by January to February 2027 due to the compute cost).
Source code and run instructions: the GitHub repository below, folder `SystemCode`, with `SETUP.md` for a 5 command setup.
Repository: https://github.com/DevelopingGod/IRS-PM-2026-08-17-AIS08FT-GRP-MANTIS-CivicOperations

Team and contributions

The four of us worked from Singapore and Saudi Arabia. From the project start on 5 September till date of this report submission we held 14 MS Teams meetings of over an hour each, kept a shared tracker, and agreed every component's inputs and outputs in a written specification before building. Appendix D is the log book.

Member	Work items
Khalid Alhazmi	Urgency (forward chaining rules over hazard and service impact knowledge bases) and Prioritisation (weighted multi criteria ranking with official NYC SLA deadlines). The Prioritisation guardrails, built 13 to 16 October: a starvation floor, ageing, holds for emergencies and duplicates, escalations that count only once approved, and a queue position that never authorises dispatch, with 15 behaviour tests. An urgency learning experiment across four model families. 33 commits.
Mohammad Alharthi	Agency and skill routing (v1.1.0), External factors (OpenStreetMap proximity, v1.0.0) and Crew deployment and route optimisation (genetic algorithm with A* routing, v1.0.0), each with its own research report and test suite. The GPU run of the language model experiments. Independent label check.
Nabeel Taresh	Duplicate detection (hybrid evidence matcher tuned on 295 hand checked pairs), Fake request detection (calibrated rules and logistic regression) and Admin questions (natural language questions over the live queue, read only). The first data flow diagram. 13 commits.
Indish Sankalp Sandeep	Intake bot, Emergency check, Credibility and escalation, Predictive alerting. The data pipeline, component specification and team guides. The application: backend, four portals, accounts, integration of all twelve components, Docker, testing and measurement. 86 commits.

Table 1. Work Distribution

Contents

Section	Title	Page
	Acknowledgement	2
	Executive summary	2
	Team and contributions	3
1	**Business case, literature review and market research**	8
1.1	The problem	8
1.2	Literature survey	8
1.3	Market review	9
1.4	Value and who benefits	9
1.5	Objectives and scope	9
2	**System design and models**	10
2.1	Architecture	10
2.2	How a request flows	11
2.3	The twelve components	11
2.4	Design decisions	12
2.5	Data	12
2.6	Privacy, security and safety	12
3	**System development and implementation**	13
3.1	Technology stack	13
3.2	The four portals	13
3.3	Planning and delivery	14
3.4	What integration taught us	15
3.5	Proposal against delivery	16
3.6	Hosting: free plans were too small, so we used our own server	16
4	**Experiments**	17
4.1	Results at a glance	17
4.2	Intake bot	18
4.3	Emergency check	18
4.4	Credibility and escalation	19
4.5	Predictive alerting	20
4.6	Duplicate detection	20
4.7	Fake request detection	21
4.8	Admin questions	22
4.9	Agency and skill routing	23
4.10	External factors	23
4.11	Urgency and Prioritisation	24
4.12	Crew deployment and route optimisation	24
4.13	The experimentation journey	26
4.14	Does each component matter? Switching them off	26
4.15	Testing	27
5	**Findings and discussion**	28
5.1	What worked	28
5.2	Limitations	28
5.3	Ethics	28

Section	Title	Page
5.4	Future work	28
6	**Conclusion**	29
	References	30
	Glossary	31
Appendix A	**Project proposal**	32
Appendix B	**Functions mapped to MR, RS and CGS**	39
Appendix C	**Installation and user guide**	40
Appendix D	**Log book**	45

List of figures

Figure	Title	Page
1	Architecture: four portals, one API, twelve components in the order a request meets them	10
2	One request from report to closure, in nine steps	11
3	Request types in the real 311 sample	12
4	Resident portal: the home page with the 3D city (left), and reporting in plain words to the Intake bot (right)	13
5	Tracking a request with the status bot (left), and the Emergency check stopping a report (right)	13
6	City staff: the ranked queue, each card saying why the AI ranked it (left), and a request's reasons step by step (right)	14
7	A crew's route on a street map from its depot, sent automatically and open to staff override (left), and the tech team's component switches (right). Map data © OpenStreetMap contributors.	14
8	Planned and actual timeline, including the Prioritisation guardrails	15
9	Commits per member per week on GitHub, all branches. Mohammad delivered his packages as files, so his work shows as deliveries rather than commits.	15
10	How much each component was tested: methods compared (left) and experiment runs (right)	17
11	Intake bot: request type accuracy on two test slices (left), slot span F1 against the pattern baseline (right)	18
12	Emergency check: recall and false alarms across thresholds (left), recall by emergency category (right)	19
13	Credibility: score against each reporter's hidden honesty (left), and fairness across report types (right)	19
14	Predictive alerting: error of each forecasting approach (left), storm day spikes caught (right)	20
15	Duplicate detection: F1 of each method on the same 295 hand checked pairs	21
16	Fake request detection: AUC of each method on 5,390 held out requests	22
17	Admin questions: three strategies on 74 gold questions	22
18	External factors: threshold candidates on validation data (left), requests flagged across the city (right). Map data © OpenStreetMap contributors.	23
19	Urgency: macro F1 of each learned model (left), and the maximum points each factor adds in Prioritisation (right)	24
20	Crew deployment: the genetic algorithm against two greedy baselines, capacity factor 1.0	25
21	Crew deployment: methods across crew capacity (left), and one planned day of routes on real roads (right). Map data © OpenStreetMap contributors.	25
22	Queue order with each component on its safe default, as Kendall tau b against everything on	26

List of tables

Table	Title	Page
1	Work Distribution	3
2	What exists and what does not	9
3	Value to each user	9
4	Components, techniques and owners	11
5	Decisions that shaped the system	12
6	Technology stack, with the reason for each choice	13
7	Integration problems found and fixed	15
8	What the proposal committed to, and what was delivered	16
9	Headline results per component	17
10	Intake bot: 6 methods compared in 64 runs, on 308 hand written messages, two slices	18
11	Intake bot (address and time): 3 methods compared in 4 runs, on 308 messages marked by hand	18
12	Emergency check: 2 methods compared in 18 runs, on hand written emergency and look alike lines	19
13	Credibility and escalation: 2 methods compared in 2 runs, on 1,121 synthetic reporters with hidden honesty	19
14	Predictive alerting: 3 methods compared in 19 runs, on daily demand with weather, held out days	20
15	Duplicate detection: 7 methods compared in 222 runs, on 295 real pairs checked by hand	21
16	Fake request detection: 4 methods compared in 5 runs, on 5,390 held out real requests	22
17	Admin questions: 3 methods compared in 6 runs, on 74 gold questions written before any strategy ran	22
18	Agency and skill routing: 4 methods compared in 4 runs, on 49,558 historical NYC 311 records	23
19	External factors: 2 methods compared in 5 runs, on historical requests and 4,445 OpenStreetMap facilities, train, validation and holdout	23
20	Urgency: 5 methods compared in 5 runs, on 672,041 policy labelled rows	24
21	Prioritisation: 1 method compared in 15 runs, on 15 behaviour tests	24
22	Crew deployment and routing: 5 methods compared in 1,310 runs, on 50 held out episodes x 5 rosters x 5 seeds, 60 small exact cases, 500 routing pairs	25
23	Experiments that changed a decision	26
24	Terms used in this report	31
25	System functions against the course knowledge and techniques	39
26	Log book: work by date	45
27	Log book: team meetings	46

1 Business case, literature review and market research

Millions of requests, 4 decisions each, and almost no reasons given. That is the problem we set out to fix.

1.1 The problem

311 lines exist so routine problems never compete with life safety calls, which is a split most large American cities have kept since the late 1990s. The volume of requests is the difficulty. Our own sample of NYC Open Data population holds 49,558 requests from five boroughs over a few weeks [1]. Each needs 4 decisions: is this really an emergency, which agency owns it, how urgent is it compared with everything else, and which crew should go, in what order.

Getting any of these wrong costs money or safety: a wasted crew trip to a duplicate or an unfounded report, a broken pavement outside a school left behind newer and easier jobs, or a crew crossing the borough twice in a day.

Residents feel the second cost. Once a request is filed it usually disappears into a queue they cannot see, so the same problem gets reported again and again, which adds load to the very queue that is slow.

1.2 Literature survey

Five threads of earlier work shaped what we built, and each changed a design decision.

311 data is not neutral. Minkoff found that 311 contact in New York varies between neighbourhoods with their social and economic make up [3]. Kontokosta et. al. showed that complaint behaviour differs across socio spatial groups, so decisions driven by complaint counts alone can be unfair [4]. Wang et. al. found that the mix of requests is a signature of a place [5], and O'Brien studied 311 reporting as a form of care for shared spaces [6]. **What we did with it:** a request rises for what it is and where it is, not for how loudly it is reported. Extra reporters count as distinct people with a cap, and credibility corrects for each request type's base rate, so honest people who report hard to confirm problems are not marked down.

High stakes decisions should be interpretable. Rudin argues that where an interpretable model performs as well, it should be used instead of explaining a black box afterwards [7]. Amershi et. al. give guidelines for human and AI interaction, among them making clear why the system did what it did and making it easy to correct [8]. **What we did with it:** rules where the data is policy (urgency, agency), the reasons beside every ranked request, and a person able to override any route.

Language understanding on a small budget. Transformer encoders [9] and their compact distilled versions [10] made sentence embeddings practical on a CPU, and Sentence BERT trains them so that sentences with similar meaning sit close together [11].

What we did with it: a small embedding model with logistic regression, competing with a fine tuned transformer.

Grouping, trust and calibrated risk. Density based clustering, DBSCAN [12] and HDBSCAN [13], groups nearby events without fixing the number of groups in advance. The Beta reputation system scores trust from counts of good and bad outcomes [14]. Calibrated probabilities matter whenever a score is shown to a person [15]. **What we did with it:** clustering was one of eight duplicate methods compared, credibility is a Beta model, and Fake request detection is calibrated.

Planning and forecasting. Assigning and ordering jobs for crews is a vehicle routing problem, hard to solve exactly at city scale [17], and genetic algorithms are among the strongest practical methods for it [18]. A* search finds shortest paths guided by an estimate of the remaining distance [19], and landmarks sharpen that estimate (ALT) [20]. Gradient boosting is a strong learner for count data such as daily demand [16]. **What we did with it:** the crew planner and Predictive alerting.

1.3 Market review

What we reviewed	What it does	What is missing
311 CRM platforms: SeeClickFix (CivicPlus), CitySourced (Granicus), Accela [22]	Multi channel intake, category routing, service level timers for escalation	No reasoning over a request's context, no explanation to staff or residents, no route optimisation
Academic 311 studies [3, 4, 5]	Explain who reports, or classify and forecast requests, one task at a time	None chains prediction into policy rules, screening, ranking, crew dispatch and a resident status loop
Vehicle routing research [17, 18]	Efficient routes for a fixed set of jobs	Ignores the priority a city assigns, and replans without regard to work a person already decided

Table 2. What exists and what does not

The gap is the chain. Each piece exists somewhere, but no system we found joins them so that each decision feeds the next and every one can be explained. That chain, with a person in charge of the risky steps, is what Civic Operations adds.

1.4 Value and who benefits

User	Value	How it shows in the system
Residents	Know their report was heard, where it is, and why	A tracking page and a status bot that answers in plain words
City staff	A ranked queue with reasons, and routes already with the crews	Every rank says why, routes are drawn on street maps, any job can be pulled back
Crews	Fewer wasted trips and a sensible order of stops	Duplicates linked, unfounded reports flagged, optimised routes from the depot
The city	Faster fixes for what matters most, measurable decisions	Every component call timed, switchable and measured

Table 3. Value to each user

1.5 Objectives and scope

- Build a runnable system, not a set of notebooks: four portals, one API, every component live.
- Keep a person in control of every decision with a real cost: escalations, impact on sensitive sites, and any change to a crew's route.
- Make every decision explainable: each component returns its reasons, and staff see them beside the result.
- Measure each component on its own and inside the system, and report the result honestly, including what did not work.
- Out of scope: live integration with a real city and real resident identities (all accounts are pseudonymous). Section 3.5 compares the proposal with what was delivered.

2 System design and models

Twelve components built by four people in two countries had to behave as one service. The design is what made that possible.

2.1 Architecture

Four portals share one API. The API runs the twelve components in four stages, each component behind an adapter that renames shared fields, calls the owner's code, and falls back to the safe default agreed in the component specification when the code is missing, fails, returns the wrong shape or is switched off.

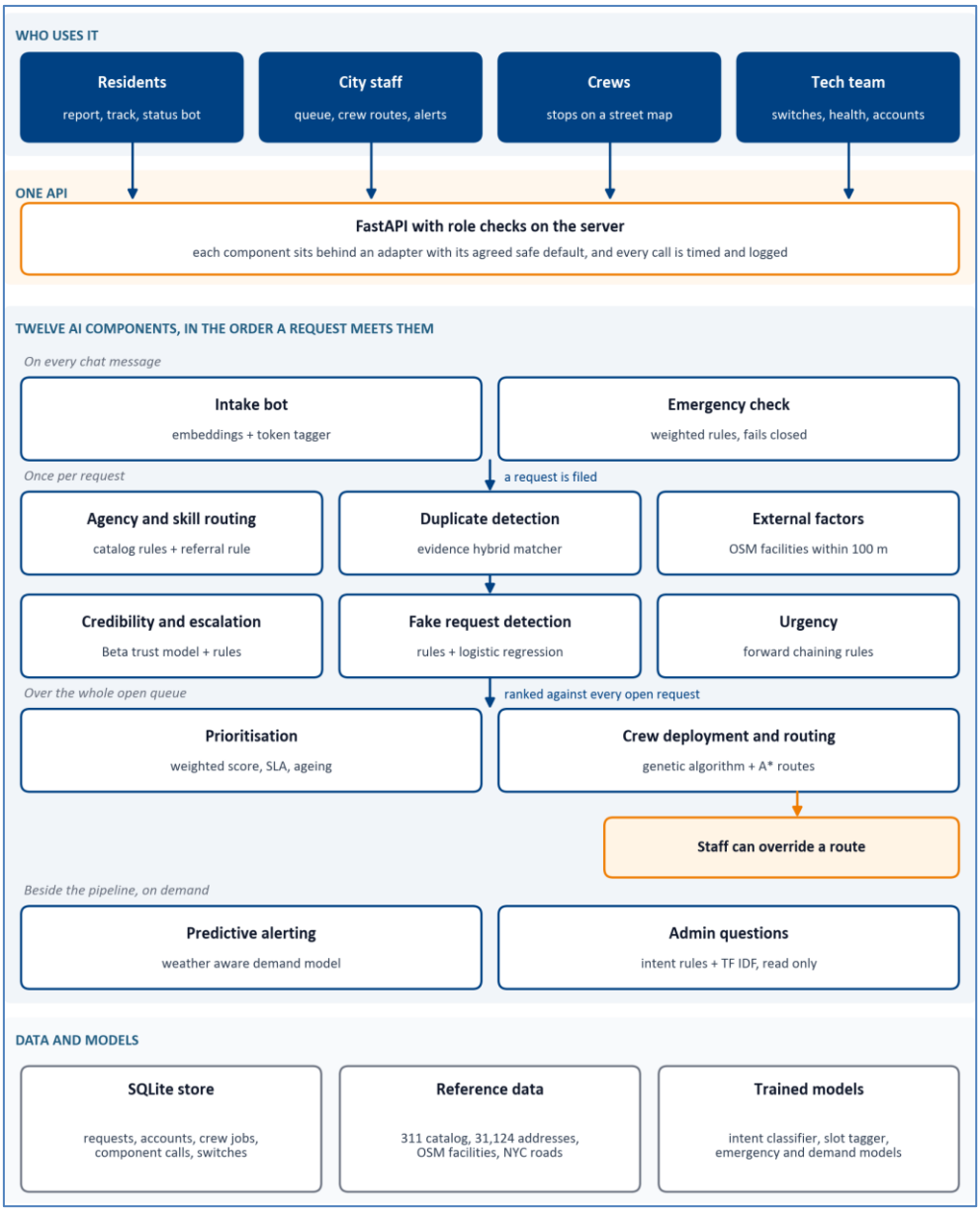


Figure 1. Architecture: four portals, one API, twelve components in the order a request meets them

To be read top to bottom: who uses it, the one API, the components in four stages, then the data. Orange marks where a person decides.

2.2 How a request flows

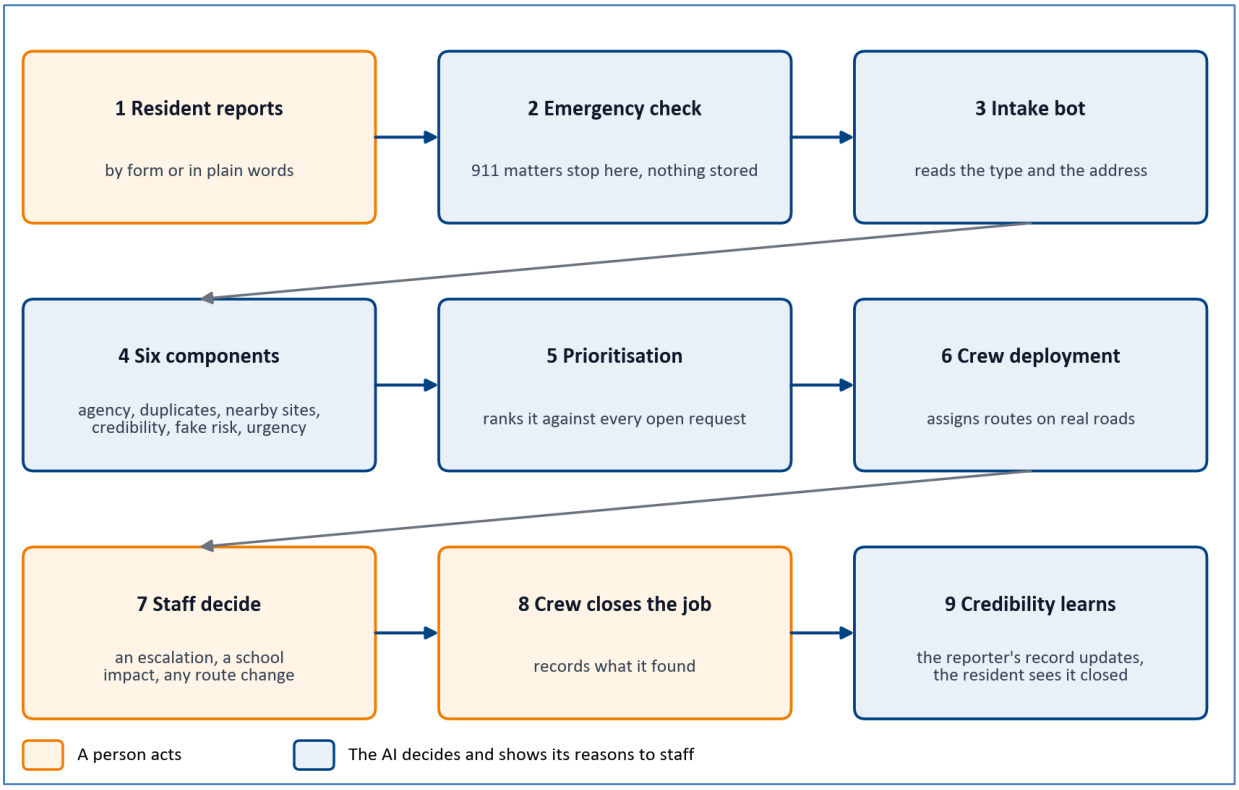


Figure 2. One request from report to closure, in nine steps

To be read left to right, row by row. People act at the orange steps and the AI at the blue ones, always showing staff its reasons.

2.3 The twelve components

Component	What it decides	Technique	Owner
Intake bot	Request type and address from plain words	Sentence embeddings with logistic regression, fine tuned token tagger for slots	Sankalp
Emergency check	Stop anything that belongs with 911	Weighted lexicon rules with a confirmation question, fails closed	Sankalp
Agency and skill routing	Agency, crew skill, hand off advisory	Catalogue rules plus a train derived referral rule	Mohammad
Duplicate detection	Same incident as an open request, or a recurrence	Hybrid evidence matcher over time, distance and address	Nabeel
External factors	A school, hospital or similar within 100 m	OpenStreetMap facilities, geodesic distance rules	Mohammad
Credibility and escalation	Reporter track record, escalation advice	Beta distribution trust model, rule engine	Sankalp
Fake request detection	Risk that a report is unfounded	Calibrated rules and logistic regression	Nabeel
Urgency	Critical, high, medium or low	Forward chaining rules over hazard knowledge bases	Khalid
Prioritisation	Place in one ranked queue	Weighted multi criteria score with SLA deadlines, ageing and guardrails	Khalid
Crew deployment and routing	Which crew, in what order, on which roads	Genetic algorithm, A* search with ALT landmarks	Mohammad
Predictive alerting	Which report types will rise on a day	Weather aware demand model against a seasonal baseline	Sankalp
Admin questions	Answers to staff questions about the queue	Intent rules and TF IDF classifier, read only queries	Nabeel

Table 4. Components, techniques and owners

2.4 Design decisions

Decision	Why
One adapter and one safe default per component	Four people built components separately. The contract let each one plug in, and a failure never stops a resident or a crew.
People decide the risky steps	An escalation and a sensitive site impact count in the ranking only once a person approves them. The AI proposes, a person decides, and the decision is recorded.
Routes go to crews automatically, staff can override	Approving every route by hand made dispatch the slowest step. Now each planned route is sent at once, a job staff pull back is never sent again by itself, and switching the component off returns to hand assignment.
An SLA for every request type	Khalid's ranking weighs the official NYC deadline. 29 of the 34 request types map to a published deadline, and the other 5, which NYC publishes none for, use a team target clearly labelled as such.
Rules where the data is the rules	The urgency learning experiment reached near perfect scores because it imitated the catalogue policy, so the explainable rules ship [7].
Pseudonymous accounts	Residents register with an email only and receive an ID. Only a keyed hash of the email is kept.

Table 5. Decisions that shaped the system

2.5 Data

Real data: 49,558 NYC 311 requests across the five boroughs from NYC Open Data [1], 31,124 distinct addresses built from them, 4,445 facilities from OpenStreetMap and the NYC drive network for routing [2], daily weather, and Khalid's table of 3,563 official NYC 311 service deadlines. Synthetic data, always labelled as such: reporter identities and their histories, crew rosters with depots, and hand written test sets (308 intake messages, 74 Admin questions, 295 duplicate pairs labelled by hand). The app starts from a real backlog: the 30 most recent requests in each borough, run through every component.

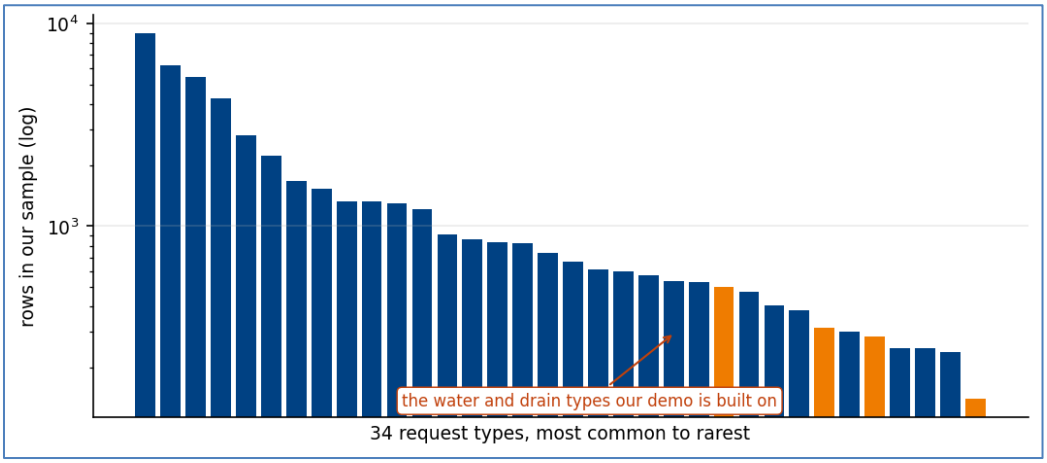


Figure 3. Request types in the real 311 sample

A few request types dominate and many are rare, so every evaluation reports macro scores that weigh rare types equally.

2.6 Privacy, security and safety

- Emergencies are never filed: the conversation stops with a full screen instruction to call 911, and no words are stored.
- Four roles, checked on the server: residents, city staff, crews and the tech team. Staff PINs are PBKDF2 hashes, and five wrong PINs lock an ID.
- Staff never see who reported a request, only the reporter's credibility score. A resident never sees a score.
- Security scans before each merge: bandit, pip audit, npm audit and gitleaks, all stay clean.

3 System development and implementation

A design is a promise. This chapter is about keeping it, and about the places where reality pushed back.

3.1 Technology stack

Layer	Choice	Why
Components	Python 3.10, scikit learn 1.7, sentence transformers 6.1, PyTorch 2.14 (CPU), pandas 2.3, SciPy 1.15, GeoPandas 1.1	Every owner's code ran in the same versions, so integration needed no rewrites
API	FastAPI 0.141 with role based access, SQLite	Typed contracts and automatic validation, no database server to run
Portals	React 19 and Vite 8, a 3D city with three.js 0.186, phone first layout	One code base for every screen size
Maps and routing	OpenStreetMap drive network, A* with ALT landmarks	Real streets for planning and for the zoomed maps
Deployment	Two Docker containers (API and nginx), one command start	The same result on any machine with Docker
Testing	pytest, Playwright 1.63 at phone, tablet and desktop size	Every role and every screen checked on every change
Security	bandit, pip audit, npm audit, gitleaks	Free, open source, run before each merge

Table 6. Technology stack, with the reason for each choice

3.2 The four portals

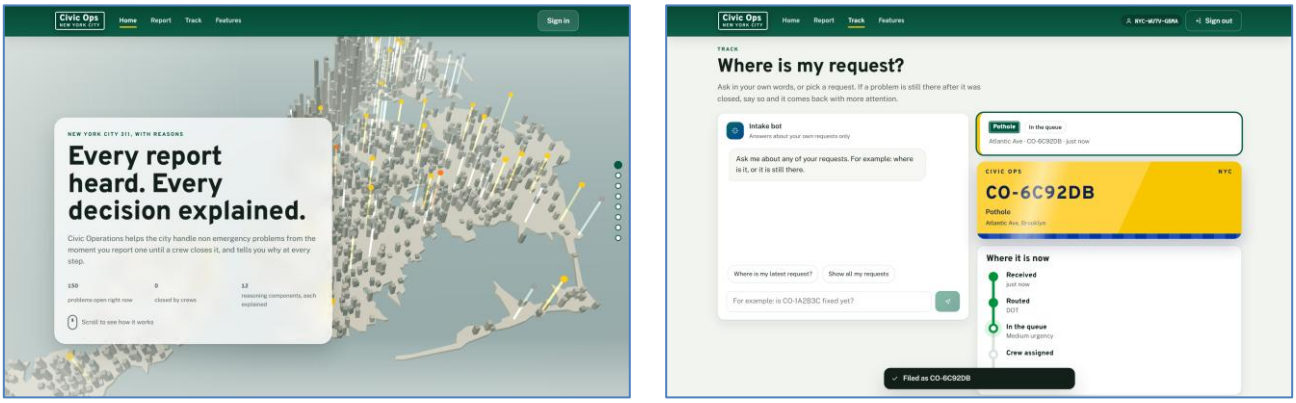


Figure 4. Resident portal: the home page with the 3D city (left), and reporting in plain words to the Intake bot (right)

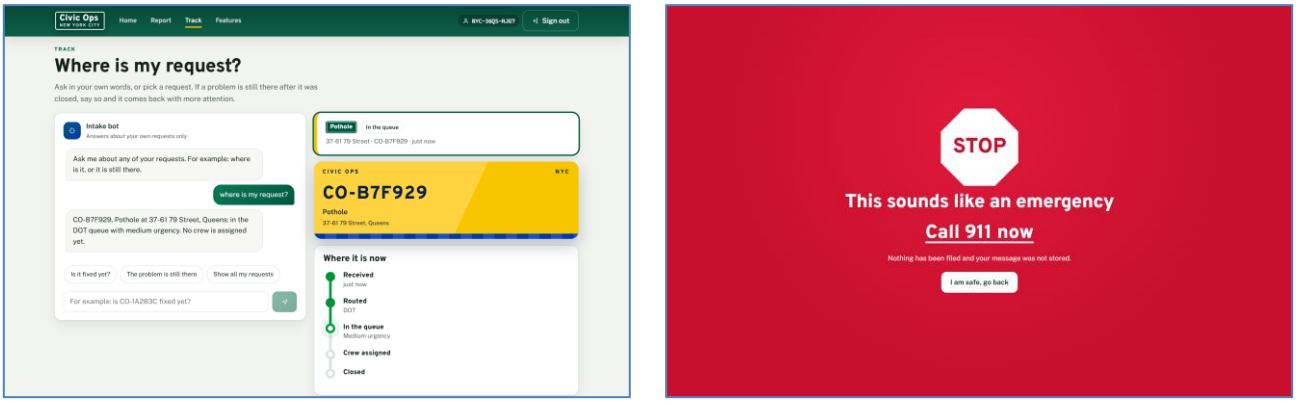


Figure 5. Tracking a request with the status bot (left), and the Emergency check stopping a report (right)

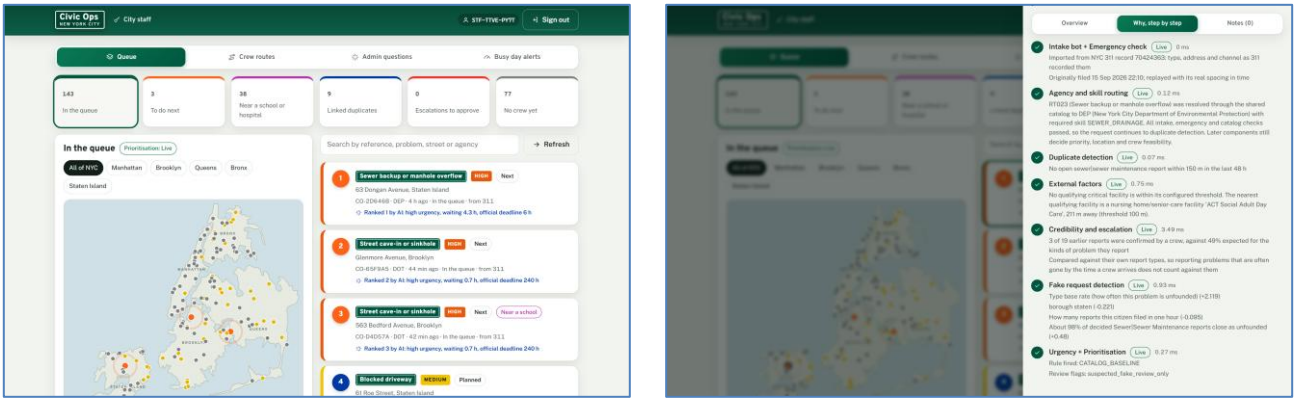


Figure 6. City staff: the ranked queue, each card saying why the AI ranked it (left), and a request's reasons step by step (right)

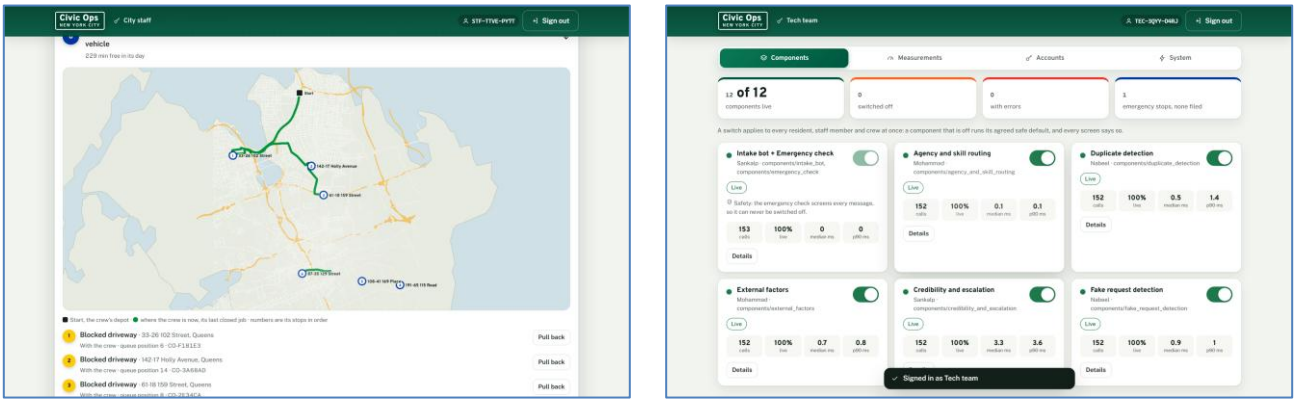


Figure 7. A crew's route on a street map from its depot, sent automatically and open to staff override (left), and the tech team's component switches (right). Map data © OpenStreetMap contributors.

3.3 Planning and delivery

We planned in three phases: define and propose (5 to 12 September), build and evaluate components separately against a written contract (to mid October), then integrate, test and measure.

The timeline below compares the planned windows from our tracker with the delivery dates in our git history. Components arrived one to ten days after their planned dates, and the contract made late arrivals cheap: each one plugged into an adapter already running on its safe default.

One task was added during the build. After a working session on 13 October, Khalid built the Prioritisation guardrails, delivered on 16 October. They make the ranking safe to act on: a request whose official deadline has passed, or one with no deadline that has waited a week, is never ranked below the NEXT band. Old requests gain ageing points after 48 and 96 hours. Emergencies and held duplicates never take a queue position. An escalation counts only once a person approves it. The ranking never trusts an urgency or deadline sent by a caller, and a queue position never authorises dispatch by itself. Fifteen behaviour tests cover them.

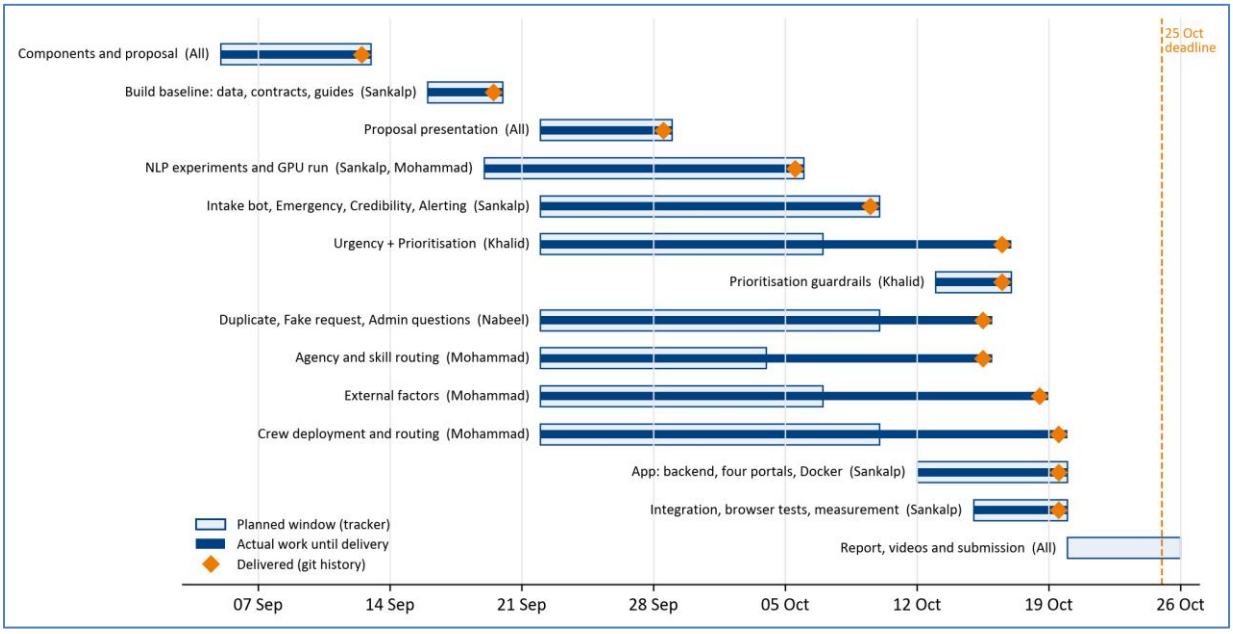


Figure 8. Planned and actual timeline, including the Prioritisation guardrails

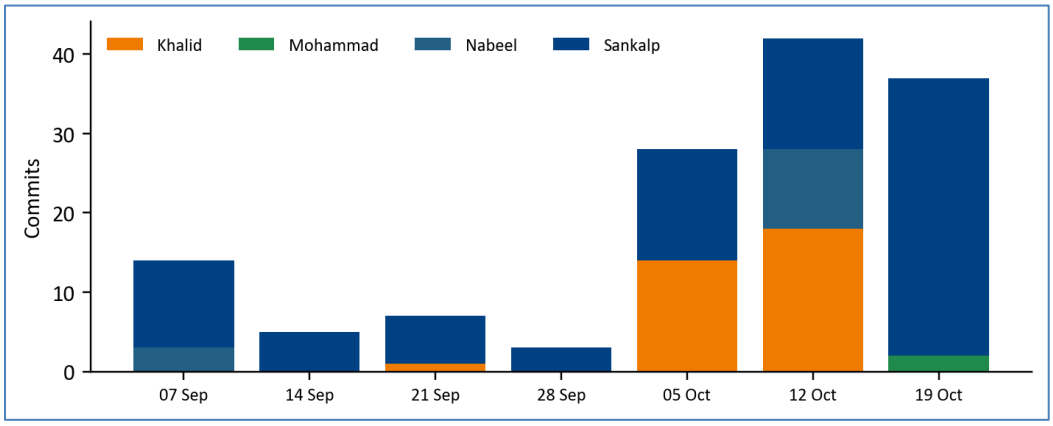


Figure 9. Commits per member per week on GitHub, all branches.
Mohammad delivered his packages as files, so his work shows as deliveries rather than commits.

3.4 What integration taught us

Joining twelve components built by four people exposed problems no component test could see. Each was traced to a root cause, fixed, and given a test so it cannot return.

What we saw	Root cause	Fix
Two reports of the same pothole were not linked	The Intake bot cut 450 Atlantic Avenue to 450 Atlantic	Taught the address reader New York street types and corners. Hand labelled addresses went from 28 to 29 of 29
Every request showed two reporters	Requests reached Duplicate detection before they had an ID, so all shared one incident group	Give each request its ID first. Now only the real duplicates show two reporters
External factors never moved the queue	Prioritisation counts a sensitive site only once its impact is confirmed, and staff could not confirm it	A confirm button for staff, like the escalation approval
Approving routes by hand slowed dispatch	Every route waited for a person, even routine ones	Routes go to crews at once. Staff pull a job back or give it to another crew
Most request types had no SLA	Only reviewed water and sewer types mapped to the official table	Mapped every type: 29 official deadlines, 5 labelled team targets
A fresh clone refused to plan crews	Windows line endings changed a file the crew planner checks by hash	Store the package byte for byte. Tested on a fresh clone

Table 7. Integration problems found and fixed

3.5 Proposal against delivery

The proposal of 12 September (Appendix A) promised to report honestly anything not finished. This is that report.

Area	Proposal	Delivered	Why
Portals	Three: citizen, crew, admin	Four: resident, staff, crew and a tech team portal	Extra. Switching and measuring components safely needed its own role
Data	Five working areas, real NYC rows	Five boroughs, 49,558 real requests	As committed
Emergency safeguard	Stop and point to 911	Stops, fails closed, stores nothing	As committed
Intake	Form, voice or conversational bot	Form and conversational bot, no voice	Not done: voice. Time went to the address reader, which fixed missed duplicates
Profanity filter	A lexicon and classifier screen	Not built	Dropped by the team as optional, so Urgency and Prioritisation could be finished well
Resolution time	Regression on resolution time	Not built	Dropped by the team as optional, to finish the rest well. Real close times are used for the team SLA targets
Prioritisation	Weighted score and a genetic algorithm	Weighted score with SLA deadlines and guardrails	The score is explainable and testable. The genetic algorithm plans crews instead
Predictive alerting	From live weather	From the recorded weather of each day	Recorded weather lets us check forecasts against what was really filed. No live feed is wired in
Crew deployment	Batch planning with A* and a genetic algorithm	As planned, sent to crews automatically with staff override	Extra: street maps with the depot and where each crew is now
Duplicates, fake reports, credibility, urgency, external factors, admin questions	One component each	All delivered and measured	As committed
Recognition leaderboard	Medals or certificates for good reporters	Not built	Outside the triage chain, so the lowest priority
Business metrics	Backlog reduction, days to fix	Not measured	They need a live city. Section 4 measures travel, requests served and queue effects instead
Hosting	A free host, to be chosen	Live on a team member's DigitalOcean server, plain HTTP, and also runs locally in Docker	Free plans were too small. Section 3.6

Table 8. What the proposal committed to, and what was delivered

Beyond the proposal: the official NYC service deadlines in the ranking, the switch off measurement of every component, the experiment cards in the tech portal, the 3D home page, zoomed street maps, 170 automated tests with 51 browser flows, and a one command Docker setup.

Some metrics also changed to fit the evidence: credibility is judged by rank agreement with hidden honesty rather than mean error, and Prioritisation by Kendall tau b [21] inside the running system.

3.6 Hosting: free plans were too small, so we used our own server

The backend keeps the language models, the NYC road network and twelve components in memory, and measured 960 MB while running. The free plan on Render allows 512 MB, so the service would be stopped as soon as it loaded. Vercel serves only the web pages, not this backend. Hugging Face's free CPU hosting was no longer free when we checked on 15 October. A paid plan large enough costs about 25 US dollars a month, which we chose not to commit the team to for an academic project.

So Nabeel deployed it on a DigitalOcean server he already had, on 21 October, from the public repository and the same five commands as the Setup Guide in Appendix C. It runs at **http://157.230.191.175/** , on the same data as ours. Anyone with Docker can still start their own copy with one command.

It is plain HTTP, without a certificate. On laptops and desktops the address opens as typed. Some phone browsers add the ‘s’ and try https on their own, find nothing there, and show an error. On a phone, type the address with http:// in full, and if the browser offers to continue to the site without a secure connection, accept.

4 Experiments

47 methods, 1,679 runs. Several of them told us we were wrong, and those were the most useful.

Each owner evaluated their component against acceptance targets agreed before building, on data the component never trained on, and compared the chosen method with at least one simpler baseline. The system as a whole was then measured by switching components off one at a time. Every number comes from a results file in the repository (folder `03_Research`) or from the component's own research report [23].

4.1 Results at a glance

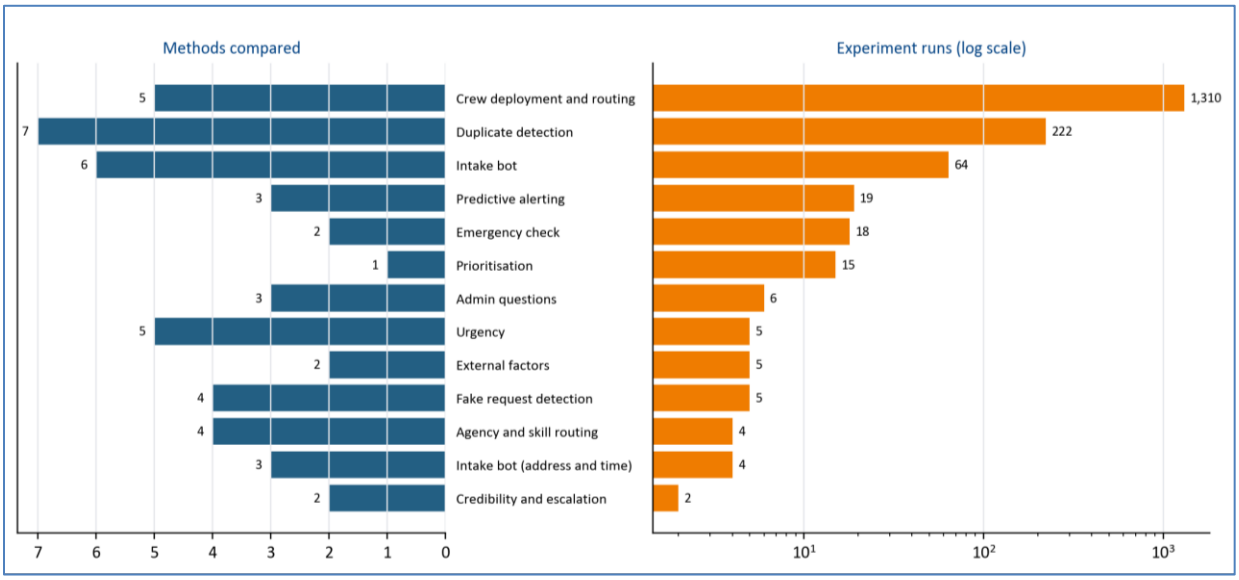


Figure 10. How much each component was tested: methods compared (left) and experiment runs (right)

Crew deployment ran the most experiments because each plan was tested over many days, crew rosters and random seeds.

Component	Test set	Result	Target or baseline
Intake bot	308 hand written messages	Macro F1 0.960, slot span F1 0.723, 78.5 ms per message	0.85 and 0.60, under 200 ms
Emergency check	Hand written emergency lines	Recall 1.000, 0 missed	Recall 1.00, no emergency filed
Credibility and escalation	1,121 synthetic reporters with hidden honesty	Spearman 0.914 over 1,121 citizens	0.85 for 10 or more reports
Predictive alerting	Daily demand with weather, held out	MAE 60.4 against 72.3, 87% of 23 storm spikes caught	Beat the no weather baseline, catch 70%
Duplicate detection	295 hand checked real pairs	Precision 0.90, recall 0.95, F1 0.92, false merges 0.10	Old method F1 0.69
Fake request detection	5,390 held out requests	AUC 0.61, false positives on first time reporters 0.33	Rules alone AUC 0.57
Admin questions	74 gold questions written before any strategy ran	Intent accuracy 0.96, unsafe requests refused 12 of 12, 3.6 ms	ML only 0.93
Agency and skill routing	49,558 historical records	Catalogue agency matched 99.996%, referral advisory for 3 codes	Learned classifiers changed 0 assignments
External factors	Historical requests, 4,445 facilities	100 m threshold, 27.3% flagged, 90 of 90 tests, 0.14 ms	Threshold from a pre declared protocol
Urgency	672,041 policy labelled rows	Learned models imitate the policy (macro F1 up to 1.00), so rules ship	Majority baseline 0.28
Prioritisation	15 behaviour tests, 34 scenarios	All pass. Every request type has an SLA target. Off, the queue reorders to tau b 0.364	Written by the owner
Crew deployment and routing	50 held out episodes, 5 rosters	9 to 18% more served and 38 to 48% less travel than rank first greedy, optimal on 60 of 60 small cases	Rank first greedy

Table 9. Headline results per component

4.2 Intake bot

The question. Can a small model read what a resident means, in their own words, fast enough to talk to them? A keyword baseline, TF IDF with an SVM, MiniLM sentence embeddings [10, 11] with logistic regression and a fine-tuned transformer [9] were trained on four variants of the training data, the transformer on a GPU. **The answer.** The embedding model won on accuracy for cost and ships: macro F1 0.960 on the team's test lines and 0.731 on 156 lines written to avoid obvious keywords. Below a confidence threshold it abstains and a person decides. A fine-tuned token tagger reads the address and time with span F1 0.723 on 308 hand marked messages, against 0.434 for hand written patterns.

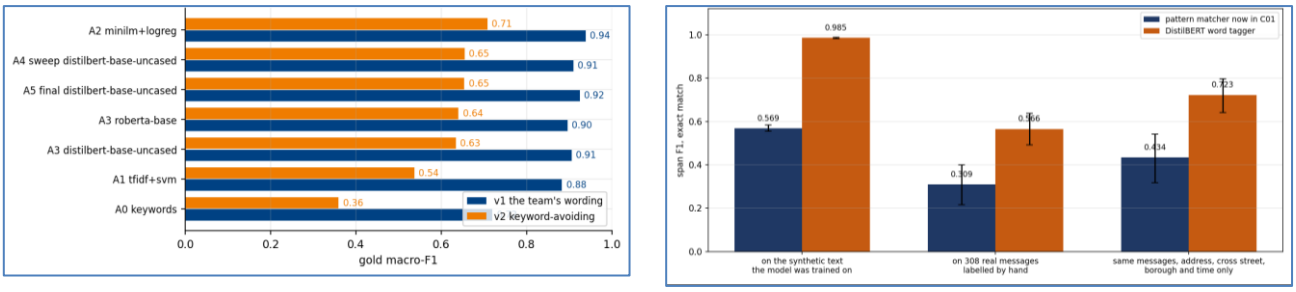


Figure 11. Intake bot: request type accuracy on two test slices (left), slot span F1 against the pattern baseline (right)

Left: the gap between the two slices shows how much an easy test set flatters a model.

Right: the learned tagger beats the pattern baseline.

Method	Runs	Result	Shipped
Keyword rules (baseline)	1	macro F1 0.543, the floor everything else has to beat	-
TF IDF with a linear SVM	4	macro F1 0.733 best, purely lexical, no sense of meaning	-
MiniLM embeddings + logistic regression	28	macro F1 0.854 on all 308 lines in this comparison. Shipped: 0.960 on the team's lines, 0.731 on keyword avoiding lines	Yes
DistilBERT fine tuned	27	macro F1 0.811 mean, best validation score, worst gold score	-
RoBERTa fine tuned	4	macro F1 0.808 best, no better than DistilBERT for twice the size	-
DeBERTa v3 small	0	skipped: sentencepiece was missing on the GPU machine	-

Table 10. Intake bot: 6 methods compared in 64 runs, on 308 hand written messages, two slices

Method	Runs	Result	Shipped
Regex patterns (the original)	1	span F1 0.434 on location and time, the floor to beat	-
DistilBERT word tagger, on the original labels	2	span F1 0.999, on template text, so it measured nothing	-
DistilBERT word tagger, labels repaired and gold spans hand labelled	1	span F1 0.723 on location and time, 0.984 on the synthetic split	Yes

Table 11. Intake bot (address and time): 3 methods compared in 4 runs, on 308 messages marked by hand

4.3 Emergency check

The question. How do we make sure no emergency is ever filed as a pothole? Weighted lexicon rules with negation handling, with a TF IDF classifier allowed only to add a detection, never remove one.

The answer. Recall 1.00: no emergency in the test set was filed. Rather than lower the threshold until hydrant and fire escape complaints raise false alarms, the bot asks the resident one question, "Is anyone in danger right now?", when unsure. If the component fails, the 911 advice is shown anyway.

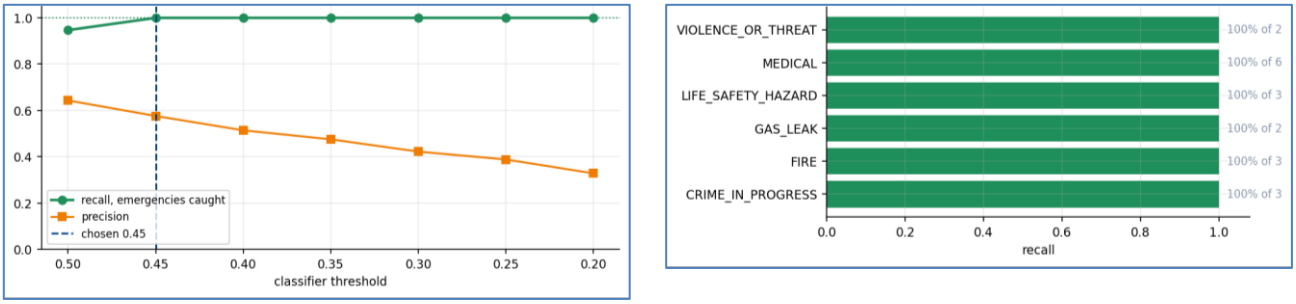


Figure 12. Emergency check: recall and false alarms across thresholds (left), recall by emergency category (right)
Left: no single threshold reaches full recall without false alarms, which is why the check asks a question instead.
Right: every category is caught.

Method	Runs	Result	Shipped
Weighted lexicon with negation	2	recall 0.737 on its own, too many emergencies missed	-
Lexicon plus TF IDF logistic regression	16	recall 1.000 at threshold 0.45, allowed only to add a detection to the lexicon, never to remove one	Yes

Table 12. Emergency check: 2 methods compared in 18 runs, on hand written emergency and look alike lines

4.4 Credibility and escalation

The question. Can we learn who reports reliably without punishing honest people? A Beta distribution trust model [14] with time decay. The same model started every reporter at a fixed neutral point, and the shipped one compares each reporter with the base rate of the request types they report.

The answer. Spearman 0.914 against each synthetic reporter's hidden honesty, and the unfair gap between hard and easy report types fell from 4.0 to 0.3 points. A rule engine recommends escalation for problems that keep returning at the same place, and a person approves it.

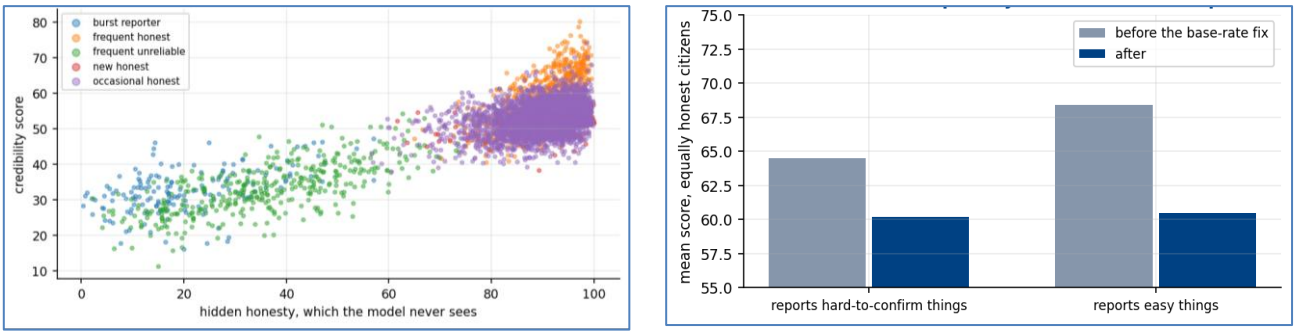


Figure 13. Credibility: score against each reporter's hidden honesty (left), and fairness across report types (right)
Left: the score rises with true honesty, so it ranks reporters correctly.
Right: after the base rate correction, report type no longer moves the score.

Method	Runs	Result	Shipped
Decaying Beta, fixed neutral prior	1	unfair by 4.0 points across report types	-
Decaying Beta, centred on the base rate for the types reported	1	Spearman 0.914 over 1,121 reporters, unfair gap 0.3 points (an earlier tuning gave 0.926 with a gap of 2.2)	Yes

Table 13. Credibility and escalation: 2 methods compared in 2 runs, on 1,121 synthetic reporters with hidden honesty

4.5 Predictive alerting

The question. Does the weather tell us which problems will rise tomorrow? Gradient boosting [16] with a Poisson loss and weather features, compared with a no weather seasonal baseline and a Poisson regression.

The answer. Count error 60.4 against 72.3 without weather, and 87% of 23 storm day spikes caught. A failure shows as failed on the dashboard, never as a quiet day.

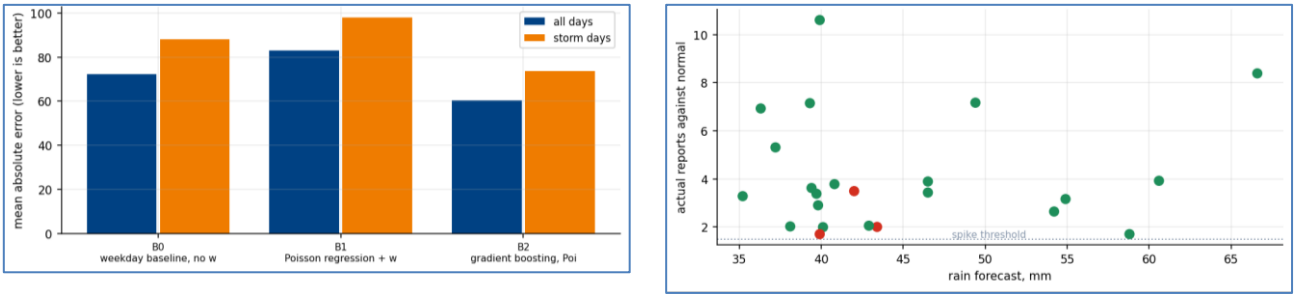


Figure 14. Predictive alerting: error of each forecasting approach (left), storm day spikes caught (right)

Left: weather features cut the error against the seasonal baseline. Right: most real spikes on storm days were flagged in advance.

Method	Runs	Result	Shipped
Weekday and season baseline, no weather	1	MAE 72.3, the honest floor	-
Poisson regression with weather	1	MAE 83.1, worse than using no weather at all	-
Gradient boosting with a Poisson loss	17	MAE 60.4 on all days, 73.7 on storm days, 87% of 23 storm spikes caught	Yes

Table 14. Predictive alerting: 3 methods compared in 19 runs, on daily demand with weather, held out days

4.6 Duplicate detection

The question. Is this the same broken thing someone already reported, or a new one next door? Seven methods were compared on 295 real pairs checked by hand: exact keys, radius and time windows, fuzzy addresses, DBSCAN [12], HDBSCAN [13], a graph method and the evidence hybrid. Figure 15 also shows the hybrid's first version and one ablation.

The answer. The tuned evidence hybrid ships: precision 0.90, recall 0.95, F1 0.92 and an adjusted Rand index of 0.91, against F1 0.69 for its first version, in 0.02 ms per request.

Exact key matching scores slightly higher, F1 0.93, so why not ship it? It needs the city's building lot number (BBL) on both reports. Many records lack one, and a report a resident files through our app never has one. The hybrid uses the BBL when it is there, which is why it matches exact key's score on these pairs, and still links reports without it by distance, time and address. A problem reported again after its request was closed is marked a recurrence, not a duplicate.

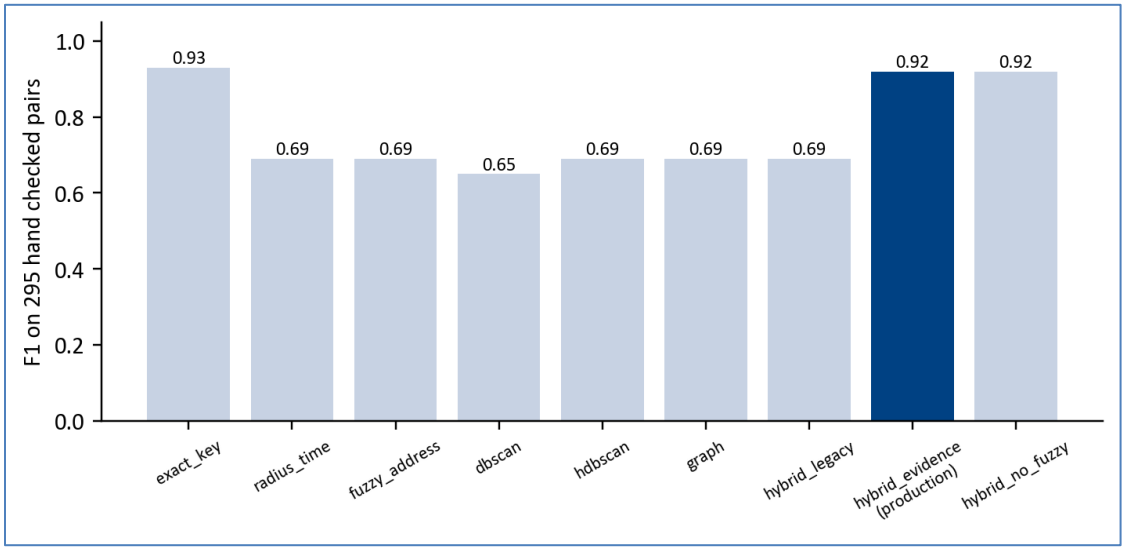


Figure 15. Duplicate detection: F1 of each method on the same 295 hand checked pairs

Clustering alone merges separate incidents. Requiring evidence from time, distance and address together nearly halves false merges.

Method	Runs	Result	Shipped
Exact BBL + type key	1	F1 0.93, false merge 0.08, but only works when the city's building lot number (BBL) is present, which resident reports lack	-
Radius + time window (150 m / 48 h)	1	F1 0.69, false merge 0.48, 150 m is too wide	-
Fuzzy address (SequenceMatcher)	1	F1 0.69, only helps weak location rows (synthetic test)	-
DBSCAN in metres + scaled hours	1	F1 0.65	-
HDBSCAN, min_cluster_size 2	1	F1 0.69, crash on many candidates fixed	-
NetworkX connected components	1	F1 0.69, transitivity chains over merge	-
Hybrid evidence (BBL + radius + fuzzy + graph)	216	F1 0.92 on the 295 hand pairs, 0.76 on a larger February sample labelled automatically, 25 m and 6 h with a per type table, works without a BBL	Yes

Table 15. Duplicate detection: 7 methods compared in 222 runs, on 295 real pairs checked by hand

4.7 Fake request detection

The question. Can a report alone tell us it is unfounded? Rules, logistic regression and gradient boosting were compared with calibration [15] on 5,390 held out requests.

The answer. Only a little. The hybrid of rules and calibrated logistic regression ships because it is explainable and keeps false positives on first time reporters at 0.33. Its separation is modest (AUC 0.61), so by design it only lowers a report or asks a person to look, and never rejects one.

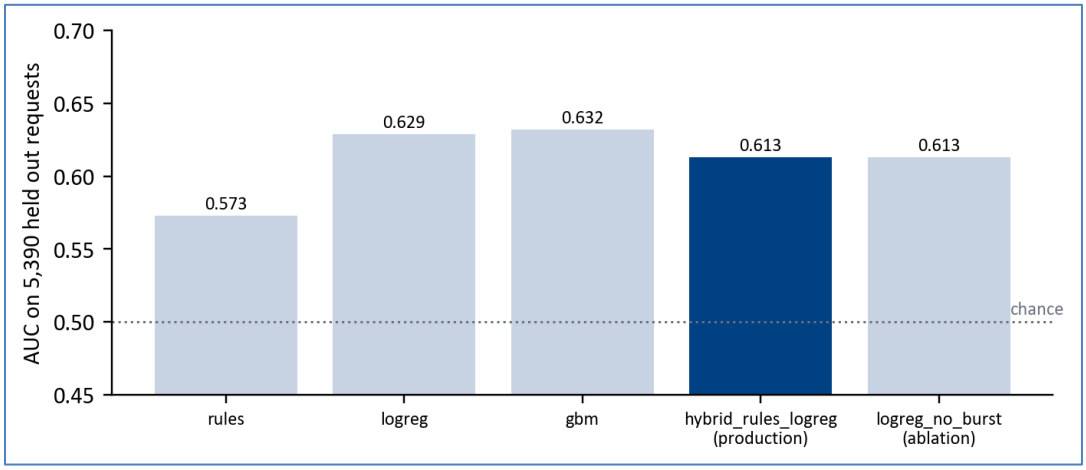


Figure 16. Fake request detection: AUC of each method on 5,390 held out requests

All methods sit just above chance, which is honest evidence that an unfounded report is hard to spot from the report alone.

Method	Runs	Result	Shipped
Rules (type base rate, burst, corroboration)	1	AUC 0.573 / Brier 0.247 on real test	-
Calibrated logistic regression	1	AUC 0.629 / Brier 0.236	-
Histogram gradient boosting	1	AUC 0.633, Brier 0.233, but flags 85% of first time reporters	-
Hybrid rules + logreg	2	AUC 0.613, Brier 0.237, first time reporters flagged at 0.33	Yes

Table 16. Fake request detection: 4 methods compared in 5 runs, on 5,390 held out real requests

4.8 Admin questions

The question. Can staff ask the queue a question in plain English and trust the answer? Rules, a TF IDF intent classifier and a hybrid were benchmarked on 74 gold questions written before any strategy ran.

The answer. The hybrid ships: intent accuracy 0.959 with confidence interval 0.91 to 1.00, every answer grounded in the data, and all 12 attempts to change data refused. It never generates SQL, so it can only read.

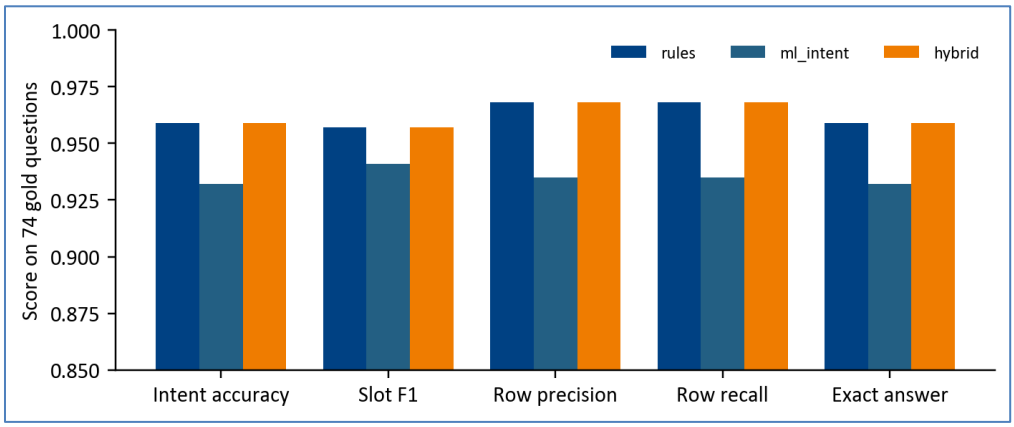


Figure 17. Admin questions: three strategies on 74 gold questions

Rules and the hybrid tie, both ahead of learning alone. The hybrid keeps the learned fallback for phrasings the rules miss.

Method	Runs	Result	Shipped
Intent rules	2	intent accuracy 0.959	-
TF IDF and logistic regression intent	2	intent accuracy 0.932	-
Hybrid, rules then the classifier	2	intent accuracy 0.959, safety 12 of 12	Yes

Table 17. Admin questions: 3 methods compared in 6 runs, on 74 gold questions written before any strategy ran

4.9 Agency and skill routing

The question. Does routing need learning at all?

The answer. No. In 49,556 of 49,558 historical requests (99.996%) the agency matched the catalogue, and learned classifiers per code and per borough changed no assignment. So the catalogue rules ship.

Learning did find one useful pattern: 503 of 1,164 referred requests fall in three codes, so the component adds an explainable advisory when a hand off between agencies is likely.

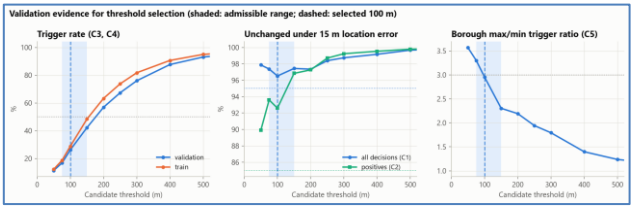
Method	Runs	Result	Shipped
Catalogue lookup by request type	1	99.996% agreement with history	Yes
Majority classifier per code	1	0 assignments changed	-
Majority classifier per code and borough	1	0 assignments changed	-
Referral advisory learned from training data	1	503 of 1,164 referrals in 3 codes	Yes

Table 18. Agency and skill routing: 4 methods compared in 4 runs, on 49,558 historical NYC 311 records

4.10 External factors

The question. Is there a school, hospital or other sensitive place close enough to matter? Straight line distance from the request to 4,445 critical facilities from OpenStreetMap [2].

The answer. A 100 m threshold, chosen on validation data under a protocol written before testing, flags 27.3% of historical requests. 2.5% of requests lack a usable location and return unknown, never zero. 90 of 90 tests pass, at 0.14 ms per request.



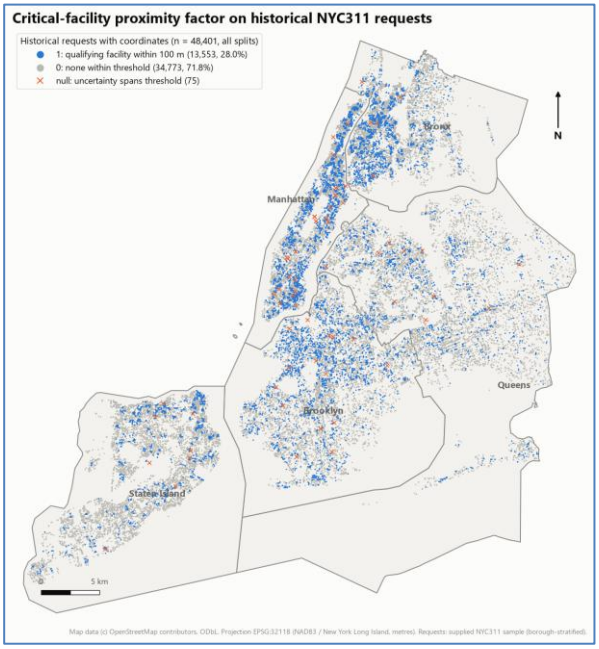


Figure 18. External factors: threshold candidates on validation data (left), requests flagged across the city (right). Map data © OpenStreetMap contributors.

Left: the candidate thresholds compared on validation data, from which 100 m was selected under the protocol. Right: where flagged requests fall across the five boroughs.

Method	Runs	Result	Shipped
Pure distance rule	3	100 m chosen under a protocol declared before testing	Yes
Context aware gating	2	available as an option, not the default	

Table 19. External factors: 2 methods compared in 5 runs, on historical requests and 4,445 OpenStreetMap facilities, train, validation and holdout

4.11 Urgency and Prioritisation

The question. How urgent is it, and what goes first? Urgency uses forward chaining rules over a hazard knowledge base and service impact rules. Four learned model families were also trained on 672,041 labelled rows and reached macro F1 up to 1.000, against 0.281 for the majority baseline.

The answer. Such a score means the models had learned the labelling policy itself, not new knowledge, so the transparent rules ship [7].

Prioritisation ranks every open request with a weighted score: waiting time, SLA risk, approved escalations, confirmed sensitive sites and extra reporters, with the guardrails of section 3.3. Its 15 behaviour tests pass. The SLA risk comes from Khalid's table of 3,563 official NYC 311 service deadlines, which his code uses once a reviewed mapping links a request type to its official category. He reviewed the water and sewer types. We mapped the rest from the most common real descriptor in our 311 sample, and matched two by hand. NYC publishes no deadline for 5 types, such as street potholes and street lights, so for those the team set a target: the median time NYC took to close that type in our sample, labelled "Team SLA target" wherever it appears. All 34 types now carry a deadline, and every ranked request says which kind it uses.

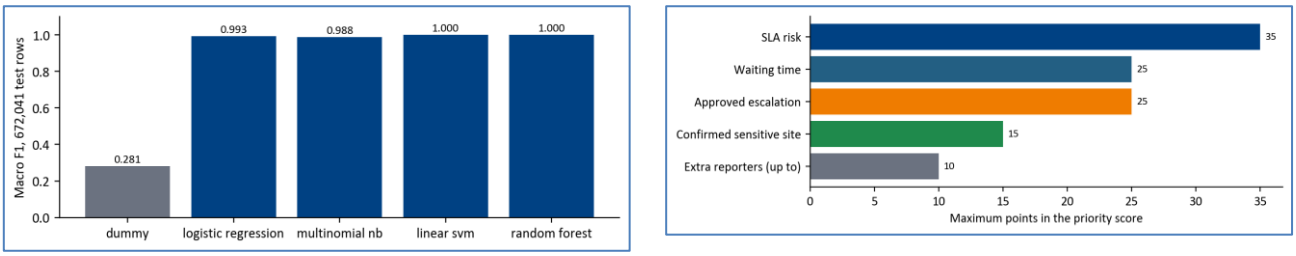


Figure 19. Urgency: macro F1 of each learned model (left), and the maximum points each factor adds in Prioritisation (right)
Left: near perfect scores reveal policy imitation, which is why rules ship.
Right: SLA risk carries the most weight, then waiting time and escalation.

Method	Runs	Result	Shipped
Majority baseline	1	macro F1 0.281	-
Logistic regression	1	macro F1 0.993	-
Multinomial naive Bayes	1	macro F1 0.988	-
Linear SVM	1	macro F1 1.000	-
Random forest	1	macro F1 1.000	-
Forward chaining rules over the hazard knowledge base	n/a	the labelling policy itself, every decision explained	Yes

Table 20. Urgency: 5 methods compared in 5 runs, on 672,041 policy labelled rows

Method	Runs	Result	Shipped
Weighted multi criteria score with guardrails	15	15 of 15 behaviour tests	Yes

Table 21. Prioritisation: 1 method compared in 15 runs, on 15 behaviour tests

4.12 Crew deployment and route optimisation

The question. Which crew goes where, in what order, without breaking the city's priorities? A genetic algorithm [18] assigns and orders jobs for each crew, with A* search [19] using ALT landmarks [20] for travel times on the NYC road network. It was compared with dispatching strictly in queue order (rank first greedy), a travel aware greedy and an exact solver on 50 held out episodes across five crew rosters.

The answer. It serves 9 to 18 percent more requests than rank first greedy with 38 to 48 percent less travel, and finds the exact optimum on 60 of 60 small cases. A travel aware greedy drives slightly less but breaks the queue's priority order far more often.

A* returned exactly Dijkstra's cost on 500 random pairs while expanding 12.8 percent of the nodes, and rolling replanning cut travel 7.6 percent against a static plan. In the app the plan is made again whenever the queue changes, each route goes to its crew at once, and jobs already with a crew stay locked to it, so routes stay stable while new work arrives.

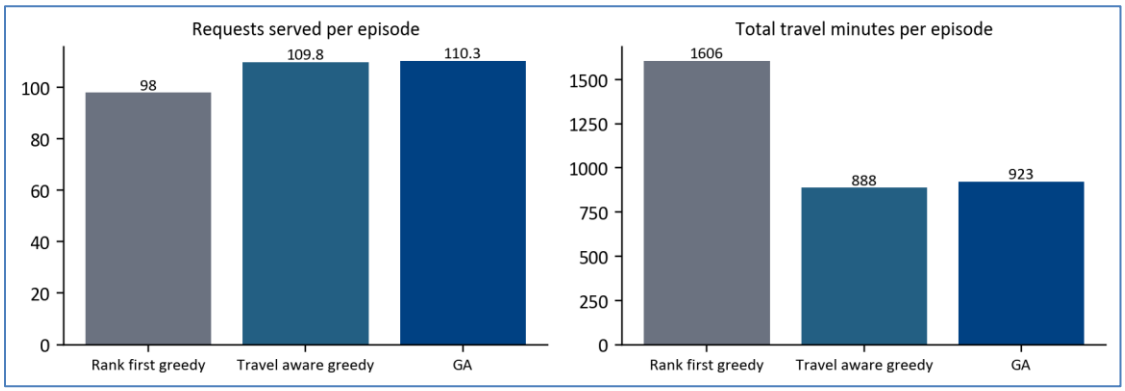


Figure 20. Crew deployment: the genetic algorithm against two greedy baselines, capacity factor 1.0

The genetic algorithm serves the most requests while keeping travel near the travel aware greedy's.

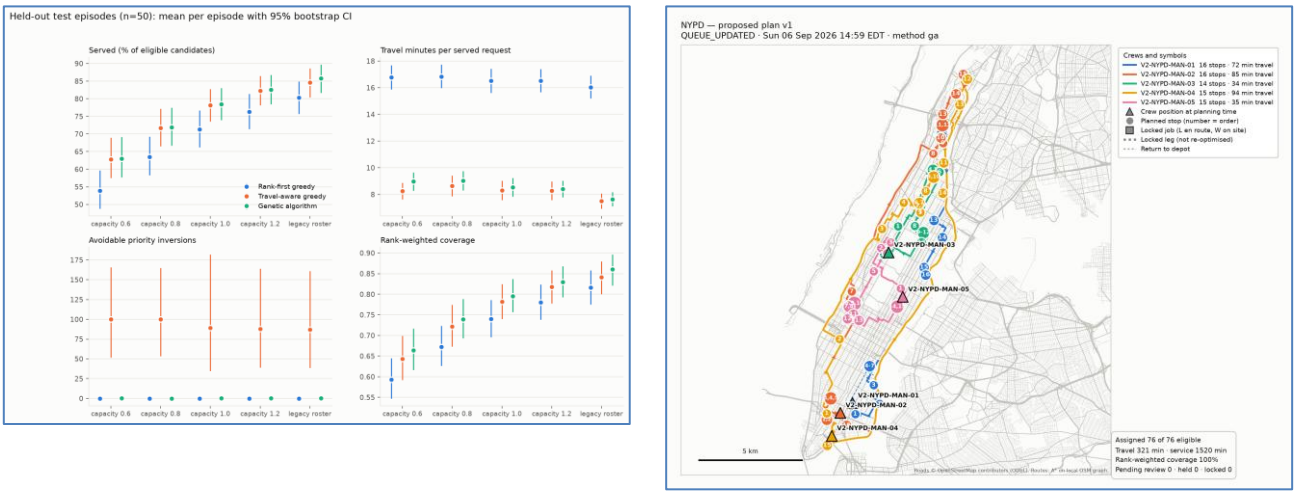


Figure 21. Crew deployment: methods across crew capacity (left), and one planned day of routes on real roads (right). Map data © OpenStreetMap contributors.

Left: the genetic algorithm's advantage is largest when crews are short. Right: each colour is one crew's ordered stops.

Method	Runs	Result	Shipped
Rank first greedy	250	the baseline	-
Travel aware greedy	250	least travel, breaks priority order	-
Genetic algorithm	1,250	best coverage with priority kept	Yes
Exact solver	60	the optimum on small cases, GA matched 60 of 60	-
Warm start insertion for replanning	50	rolling replans 7.6% less travel	Yes

Table 22. Crew deployment and routing: 5 methods compared in 1,310 runs, on 50 held out episodes x 5 rosters x 5 seeds, 60 small exact cases, 500 routing pairs

4.13 The experimentation journey

Several results changed our design. The table lists the turns that mattered, in the order they happened.

Experiment	What we found	What we changed
Intake: keyword, TF IDF, embedding and transformer models on four training variants	Sentence embeddings with logistic regression beat TF IDF and a fine tuned transformer on accuracy for cost	Shipped the small embedding model, under 80 ms per message on a CPU
Intake: our own 156 test lines that avoid keywords	Scores dropped from 0.96 to 0.73 on lines written to avoid obvious words	Reported both slices, and added a slot tagger trained on hand marked spans
Emergency: threshold sweep	Any threshold that kept recall at 1.00 also raised false alarms on hydrants and fire escapes	Ask the resident one question when unsure, instead of a lower threshold
Credibility: fairness by request type	Hard to confirm problem types cost honest reporters 4 points	Corrected for each type's base rate, now 0.3 points
Duplicates: 295 pairs checked by hand	The first matcher merged half of the separate incidents (F1 0.69)	Tuned an evidence based hybrid (F1 0.92)
Urgency: four learned model families	Near perfect scores, because the labels were the catalogue policy itself	Kept the explainable forward chaining rules
Crew planning: objective chosen on validation episodes only	A ranked objective protected priority better than a weighted sum	Froze it before testing on 50 held out episodes
System: switching each component off	At first three components left the queue unchanged	Found two integration faults and a missing staff step, then fixed them

Table 23. Experiments that changed a decision

4.14 Does each component matter? Switching them off

To see each component's effect on the whole system, we filed 34 hand written scenarios, with every fourth reported again by a second resident (43 cases), first with everything on and then with one component at a time on its safe default. A dispatcher's sign offs were applied, because Prioritisation counts escalations and sensitive sites only once a person approves them. Agreement between the two queues is measured with Kendall tau b [21].

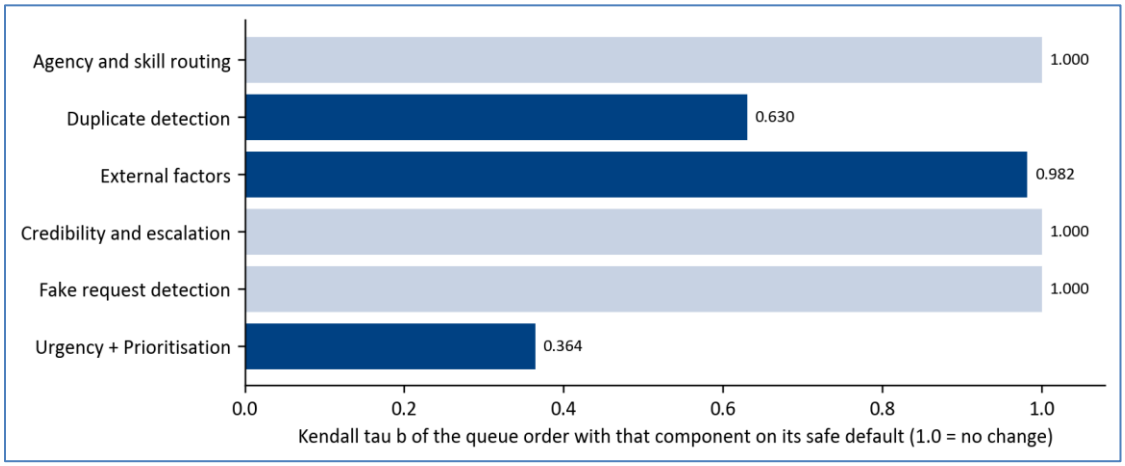


Figure 22. Queue order with each component on its safe default, as Kendall tau b against everything on

Bars below 1.0 mean switching that component off changes who is served first.

Prioritisation moves the queue most, then Duplicate detection and External factors. Agency routing does not move it, but without it nearly every case loses its dispatch decision. Credibility changes the fake risk action on many cases but not the order, because it reaches the queue only through an escalation, and no test reporter has the repeated closed reports that trigger one. Fake request detection changes its own action and never the order, by design: it flags reports for a person and never rejects one.

4.15 Testing

- 170 automated tests across the app and the Python components, plus each owner's own suites (for example 83 for Crew deployment, 90 for External factors).
- 51 browser test flows at phone, tablet and desktop size, checking every role, every screen and layout faults such as text cut off or content sticking out.
- A fresh clone from GitHub was set up from the written guide and run end to end before each release.

5 Findings and discussion

What we would tell the next team, starting with what worked.

5.1 What worked

- The contract first approach. Agreeing inputs, outputs and safe defaults before building let four people work separately and still integrate twelve components in days.
- Rules and learning where each fits. Learning won where language is messy (intake) or patterns are statistical (duplicates, demand). Rules won where the data is policy (urgency, agency), and the experiments showed why.
- Measuring inside the system. Component scores alone hid integration faults that the switch off measurement exposed.
- Automation with an exit. Sending routes automatically removed the slowest step, and the pull back button and the off switch keep a person in charge.

5.2 Limitations

- Synthetic people and crews. Reporter histories and crew rosters are generated, so credibility and crew results show the method works, not its real world value.
- Credibility does not move the test queue, because no test reporter has the repeated closed reports that make it recommend an escalation.
- Fake request detection separates genuine from unfounded reports only modestly (AUC 0.61), so it flags for review and never rejects.
- Five request types use a team SLA target, the median close time in our sample, because NYC publishes no deadline for them. A real agency would set its own.
- A crew's position is the place of its last closed job, not a live GPS fix.
- The live server is plain HTTP. PINs travel unencrypted, and on phones that switch to https on their own the address shows an error until http:// is typed in full (section 3.6).

5.3 Ethics

A system that scores residents can treat them unfairly [4]. Credibility starts every new reporter at exactly neutral, one bad report cannot drop a trusted reporter into the low band, and reporting hard to confirm problems costs about 0.3 points. Staff never see who reported a request, and the resident never sees any score.

5.4 Future work

Two pieces of work would matter most. Both were out of reach in seven weeks, for reasons no amount of effort could change.

- **A pilot with a real city agency.** Real crews, real rosters and real outcomes flowing back into credibility, with dispatch running beside the agency's own for a few months. It needs a partner agency, a data sharing agreement and an ethics approval, none of which a student project can obtain in a term, and it is the only way to measure the business metrics of the proposal: backlog, days to a lasting fix and wasted trips.
- **Learning the ranking from dispatcher decisions.** Today the ranking weights are set by hand and explained. With months of logged decisions, overrides and outcomes, the weights could be learned and checked offline against what really happened, keeping the explanation. Those logs do not exist in any open dataset, so they can only be collected by running the system for real first.

6 Conclusion

Civic Operations shows that city service triage can be automated through intelligence and be explainable at the same time. Twelve components built by four people work together as one service, each with a measured effect and a safe failure. Crews receive their routes without waiting, and people keep the decisions that carry risk, with a way to change any of them. The most useful lessons came from integration and honest measurement rather than from any single model.

References

[1] NYC Open Data. 311 Service Requests from 2010 to Present. City of New York. https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9

[2] OpenStreetMap contributors. Map data, Open Database Licence 1.0. https://www.openstreetmap.org/copyright

[3] Minkoff, S. L. (2016). NYC 311: A tract level analysis of citizen and government contacting in New York City. Urban Affairs Review, 52(2), 211 to 246.

[4] Kontokosta, C. E. and Hong, B. (2021). Bias in smart city governance: How socio spatial disparities in 311 complaint behavior impact the fairness of data driven decisions. Sustainable Cities and Society, 64, 102503.

[5] Wang, L., Qian, C., Kats, P., Kontokosta, C. and Sobolevsky, S. (2017). Structure of 311 service requests as a signature of urban location. PLOS ONE, 12(10), e0186314.

[6] O'Brien, D. T. (2018). The Urban Commons: How Data and Technology Can Rebuild Our Communities. Harvard University Press.

[7] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence, 1, 206 to 215.

[8] Amershi, S. et al. (2019). Guidelines for human AI interaction. Proceedings of CHI 2019, ACM.

[9] Devlin, J., Chang, M. W., Lee, K. and Toutanova, K. (2019). BERT: Pre training of deep bidirectional transformers for language understanding. Proceedings of NAACL.

[10] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N. and Zhou, M. (2020). MiniLM: Deep self attention distillation for task agnostic compression of pre trained transformers. NeurIPS.

[11] Reimers, N. and Gurevych, I. (2019). Sentence BERT: Sentence embeddings using Siamese BERT networks. Proceedings of EMNLP.

[12] Ester, M., Kriegel, H. P., Sander, J. and Xu, X. (1996). A density based algorithm for discovering clusters in large spatial databases with noise. Proceedings of KDD.

[13] Campello, R. J. G. B., Moulavi, D. and Sander, J. (2013). Density based clustering based on hierarchical density estimates. Proceedings of PAKDD.

[14] Josang, A. and Ismail, R. (2002). The Beta reputation system. Proceedings of the 15th Bled Electronic Commerce Conference.

[15] Niculescu Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of ICML.

[16] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189 to 1232.

[17] Toth, P. and Vigo, D. (eds.) (2014). Vehicle Routing: Problems, Methods, and Applications, 2nd edition. SIAM.

[18] Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. Computers and Operations Research, 31(12), 1985 to 2002.

[19] Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100 to 107.

[20] Goldberg, A. V. and Harrelson, C. (2005). Computing the shortest path: A* search meets graph theory. Proceedings of SODA.

[21] Kendall, M. G. (1945). The treatment of ties in ranking problems. Biometrika, 33(3), 239 to 251.

[22] SeeClickFix (CivicPlus), CitySourced (Granicus) and Accela product documentation, reviewed September 2026.

[23] Civic Operations source code, data, models and research files (this submission). https://github.com/DevelopingGod/IRS-PM-2026-08-17-AIS08FT-GRP-MANTIS-CivicOperations

Glossary

Term	Meaning
311	New York City's number and service for non emergency requests to city agencies.
A* search, ALT	A shortest path search guided by an estimate of the remaining distance. ALT uses landmarks to sharpen that estimate.
Adapter	The thin piece of code that connects a teammate's component to the app without changing its logic.
AUC	Area under the ROC curve: how well a score separates two classes. 0.5 is chance, 1.0 is perfect.
Automatic assignment	Crew deployment sends each planned route to its crew at once. Staff can pull a job back or give it to another crew.
Beta trust model	A credibility score built from counts of confirmed and unfounded reports, starting neutral.
Brier score	The mean squared error of predicted probabilities. Lower is better calibrated.
Docker container	A packaged program with everything it needs, so it runs the same on any computer.
Escalation	A recommendation to treat a recurring problem more seriously. It counts only once a person approves it.
F1, precision, recall	Recall is the share of true cases found, precision the share of flags that are right, F1 balances both.
Genetic algorithm	A search that improves a population of candidate plans by selection, crossover and mutation.
Guardrail	A rule that keeps an automated decision safe, such as a floor on how low an overdue request can rank.
Intent and slot	The kind of request a message describes, and the details it gives, such as the address or time.
Kendall tau b	Agreement between two orderings, allowing for ties. 1.0 means the same order.
MANTIS	Our team name: Machine Intelligence, Analytics, Networks, Technology and Intelligent Systems.
MR, RS, CGS	Machine Reasoning, Reasoning Systems and Cognitive Systems, the three courses this practice module draws on.
OSM	OpenStreetMap, the open map data used for facilities and roads.
Pull back	Staff take a job away from its crew. It then waits for a person and is never sent again automatically.
Safe default	The agreed output a component gives when its code is missing, fails or is switched off.
SLA	Service level agreement: the target time an agency sets for a request type.
Spearman correlation	Agreement between two rankings of the same items.
Synthetic data	Generated data, labelled as such, used where real data cannot exist, such as reporter histories.
Team SLA target	For request types NYC publishes no deadline for: the median time NYC took to close that type in our sample, always labelled.

Table 24. Terms used in this report

Appendix A Project proposal

The proposal as submitted on 12 September 2026, as editable text. Section 3.5 compares it with what was delivered.

PROJECT PROPOSAL

Name of Topic:

Civic Operations: Explainable City Service Triage and Field Crew Dispatch

NUS MTech in AIS, Intelligent Reasoning Systems

Field	Detail
Date of Proposal	12th September 2026
Project Title	Civic Operations: Explainable City Service Triage and Field Crew Dispatch
Group Number	23
Group Members	Khalid Alhazmi **[A0361128J]** Mohammad Alharthi **[A0354097W]** Nabeel Taresh **[A0352296X]** Sankalp Sandeep Indish **[A0351383E]**
Sponsor / Client	Not applicable. This is an internally driven academic project, no external sponsor is confirmed.

1. Introduction

We decide to build Civic Operations Project to help a city operations team decide which non emergency service requests to handle first, with which crew, and why. We deliver it as three connected applications over one shared backend, a citizen portal for filing requests by form, voice, or a conversational bot, a crew portal for field workers to report the true outcome of each job, and a municipality portal giving administrators a live operational dashboard. The system predicts urgency and effort, understands if any external factors (such as close proximity to locations that are critical), applies the city's own policy rules, groups repeated and cross channel reports of the same incident, screens for requests that are not really there, forecasts likely demand from live weather conditions, and assigns and routes crews efficiently, explaining every decision in plain language. Our goal is a system a real city could trust, faster resolution of genuinely urgent problems, a shrinking backlog, fair treatment of citizens, and decisions that an auditor can understand.

2. Project Background and Market Context

Non emergency complaints are handled through 311 systems, kept deliberately separate from 911 emergency lines since **Baltimore's 1996 rollout**, so routine issues such as potholes, blocked drains, and noise do not compete with life safety calls.

Existing 311 software, including SeeClickFix (CivicPlus) and CitySourced (Granicus), focuses on multi channel intake and category based routing, with escalation, where it exists, implemented as a service level timer rather than a genuine check for a recurring problem.

In researching this space, we found that none of the platforms or papers that we reviewed chain a prediction into a policy layer, into duplicate and fake screening, into crew and route optimization, into a citizen facing status loop, the way we do here. That combination is our market position.

3. Literature Review and Market Research

What Was Researched	What Was Found	What Does Not Exist Yet
SeeClickFix, CitySourced, and Accela class 311 CRMs, IBM 311 capstone projects, Scaling the Queue (arXiv 2605.06482, 2026), and an open data curation study (arXiv 2502.08649, 2025).	Commercial platforms stop at intake, routing, and time-based escalation. Every research project reviewed stops at one modelling stage, classification, resolution time prediction, or reinforcement learning based routing.	No system reviewed chains prediction into policy rules, duplicate and fake screening, crew and route optimization, and a citizen facing explanation loop together. No system reviewed keeps a recoverable, per citizen credibility score inside a municipal complaint pipeline, the closest real analog found is Facebook's hidden content moderation trust score, from an unrelated domain.

4. Project Scope

We commit to all 4 technique groups as real, working deliverables. Decision Automation runs the rule engines behind escalation, eligibility, and duplicate handling. Resource Optimization assigns crews and plans routes with informed search and a genetic algorithm. Knowledge Discovery predicts urgency, effort, and resolution time, understands if there are any external factors adding to decision and detects recurring hotspots and fake reports. Cognitive Systems is delivered as the NLP intake bot and the admin facing natural language question answering layer.

In Scope

- **Three connected applications:** a citizen portal, a crew portal, and a municipality admin portal, sharing one backend and one database.
- **Five working areas**, **not one borough alone**, so the system is demonstrably not static or single area based, using up to 10,000 real NYC 311 rows per area, topped up with synthetic data where more volume or history is needed.
- **An explicit emergency safeguard:** any request whose content suggests a medical, police, or criminal matter is stopped before filing, with a prominent on-screen instruction to contact 911 instead.

Out of Scope for This Phase

- Any live integration with an actual city agency or a real crew workforce. All crew and citizen data used for demonstration is synthetic and clearly labelled as such.
- A final, fixed choice of paid infrastructure. Hosting for the Python backend service and the database is described in Section 6 as still being evaluated among free options, and will be finalized during the build.

Note: *If we cannot complete any part of this scope by the final deadline despite this plan, we will report that honestly in the final submission's Challenges and Roadblocks section, rather than hide it or quietly drop it.*

5. Data Collection and Preparation

Our primary source is the NYC 311 Service Requests dataset, 2020 onward, drawn from 5 working areas rather than one borough, with up to 10,000 real rows pulled per area directly from NYC Open Data at no cost. We deliberately changed this from an earlier, larger single area pull, so the working dataset stays small enough to version, fast to iterate on, and genuinely multi area rather than static. **Where we need additional volume, history, or coverage beyond that real sample, we use synthetic rows and label them clearly as synthetic** throughout the system and the report, never presented as if they were real 311 records.

Synthetic Data, Generated Once and Shared

Several components need synthetic data the public dataset cannot provide, citizen and reporter identity, crew rosters, resolution time labels, and urgency scenario labels. **Every teammate's own worked upon and documented knowledge independently converges on the same requirement**, that we generate these synthetic populations once and share them across components rather than each of us regenerating our own, so that a synthetic citizen's credibility score and their duplicate or fake reporting history refer to the same person throughout the system.

6. System Design

We organize **the system as three portals over one shared reasoning pipeline**.

A citizen files a request by web form or by speaking to a lightweight NLP bot, both are screened for profanity and for emergency content before anything is stored. Every request then passes through duplicate and fake screening, urgency assignment, escalation and credibility scoring, external factors applicability checks and prioritization in near real time, while crew assignment and route optimization run periodically as a batch planning step over all open, prioritized requests.

A crew portal lets field workers report the true outcome of each job, closing the loop back to the citizen's own status view and to credibility scoring.

An admin portal gives the municipality a live operational dashboard. The full diagram is on the following page.

Component Ownership and Technique Mapping

Component	Owner	Core Technique	Technique Group
Duplicate & Cross-Channel Detection	Nabeel Taresh	Exact key match, DBSCAN/HDBSCAN, NetworkX grouping	Knowledge Discovery, Decision Automation, Cognitive Systems
Fake & Unfounded Request Detection	Nabeel Taresh	Rule guardrail, logistic regression, gradient boosted trees	Decision Automation, Knowledge Discovery
Escalation & Citizen Credibility Scoring	Sankalp Indish	Rule engine, Beta distribution trust model	Decision Automation, Knowledge Discovery
Urgency Assignment	Khalid Alhazmi	Forward chaining rules, weighted scoring	Decision Automation
Request Prioritization	Khalid Alhazmi	Weighted multi criteria score, genetic algorithm	Decision Automation, Resource Optimization
Resolution Time & External Factors	Mohammad Alharthi	Synthetic resolution time regression, OSM proximity	Knowledge Discovery, Cognitive Systems

Crew Deployment & Route Optimization	Mohammad Alharthi	A* informed search, genetic algorithm	Resource Optimization
NLP Intake Bot	Sankalp Indish	Lightweight intent and slot filling	Cognitive Systems
Advanced Predictive Alerting	Sankalp Indish	Weather cross referenced pattern matching	Knowledge Discovery
Profanity Filter	Khalid Alhazmi	Lexicon and classifier-based text screening	Decision Automation, Knowledge Discovery

Note: *Full field level inputs, outputs, business metrics, benchmarking plans, and evaluation metrics for every component above are documented in the shared component template maintained in the project repository, so the detail behind each row is not lost, only kept out of this table.*

How This Works, One Example

Picture a single request during a storm.

A citizen who cannot type tells our NLP intake bot by voice that a drain outside their home is flooding. The bot fills in the request, and the profanity filter finds nothing to flag.

The emergency check confirms this is not a medical, police, or criminal matter, so it is not redirected to 911.

Duplicate and cross-channel detection checks whether anyone nearby has already reported the same flooding, in this example nobody has, so the request stands on its own.

Escalation and credibility scoring checks the citizen's trust score, a returning citizen with a good track record starts fully trusted, so no extra scrutiny is applied, and fake and unfounded request detection finds nothing suspicious.

The external factors and resolution time component work towards understanding parameters if the request has been raised to a critical location such as an area with closer proximity to a hospital. This helps add to knowledge of resolution time and prioritization in linear, directed way.

Meanwhile, advanced predictive alerting has already told the admin dashboard, from live weather data, that heavy rain in this area makes drainage complaints likely today, so this request confirms rather than surprises that forecast.

Urgency assignment reads the storm context and the complaint type and assigns a high urgency level, and request prioritization places it near the top of today's queue.

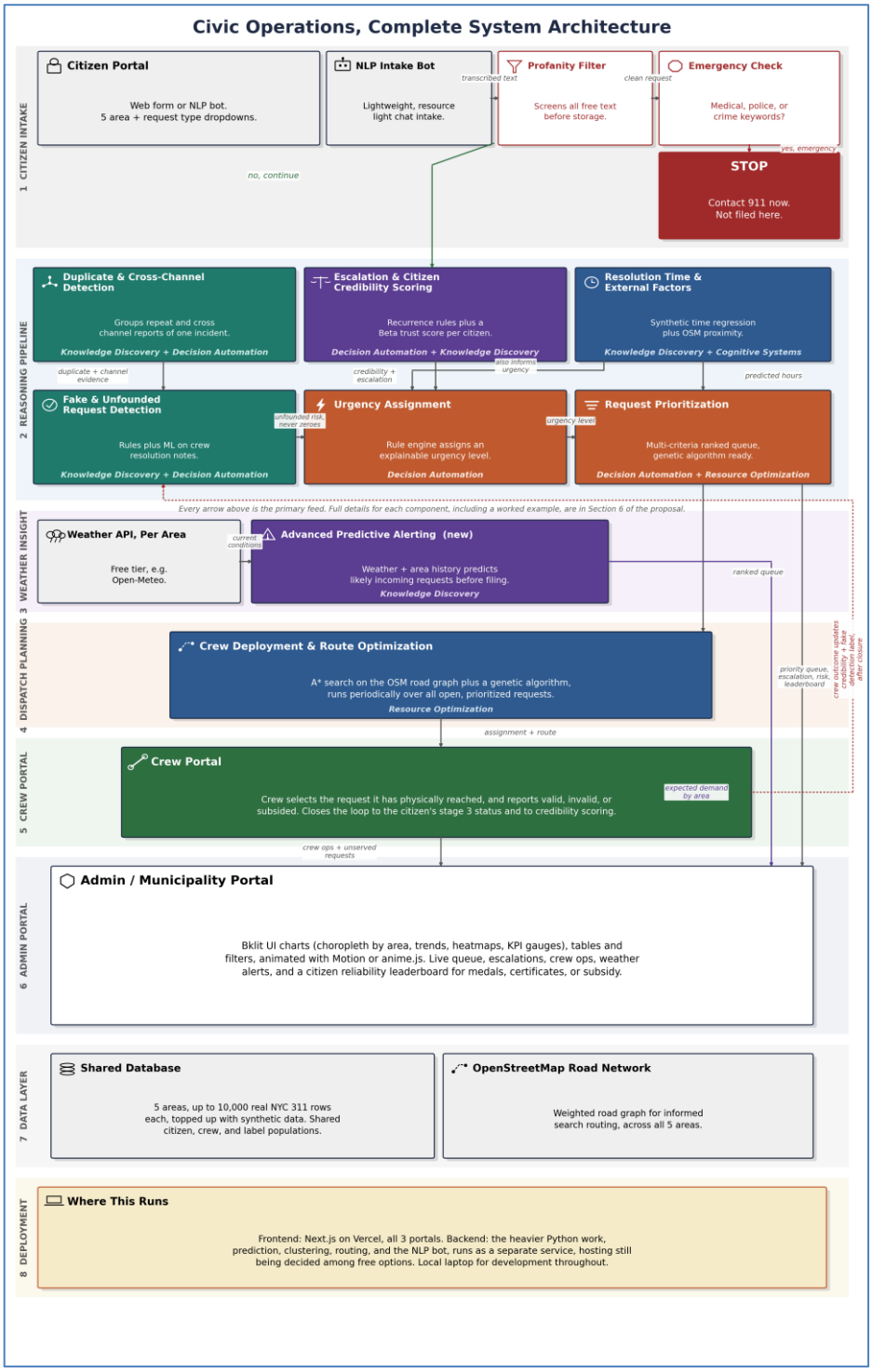
In the next batch planning run, crew deployment and route optimization assigns the nearest qualified crew and plans their route.

The citizen sees their request move to stage two, processed, with an estimated resolution time and the crew type assigned.

Once a crew clears the drain, they log the outcome as valid in the crew portal, the citizen's status moves to stage three, resolved, and credibility scoring records another confirmed genuine report, keeping this citizen's trust score high for next time.

System Architecture Diagram

Note: *A full resolution version of this diagram is also available in the project repository.*



7. Evaluation Metrics

We evaluate this system on two levels, whether each component performs well on its own technical terms, and whether the system as a whole actually helps the city. A metric is only useful if it is matched to the kind of output it is judging, so the tables below name a specific metric for each situation rather than a generic accuracy figure.

Technical and AI Metrics

Component or Task	Metric	What It Measures
Duplicate and cross-channel detection	Precision, recall, F1 score	How well true duplicate pairs are found without merging genuinely separate incidents
Duplicate and cross-channel detection	Silhouette score, Adjusted Rand Index	How well separated the spatio-temporal clusters are, and their agreement with a hand labelled sample
Fake and unfounded request detection	AUC ROC, Brier score	Separation between genuine and unfounded reports, and how well calibrated the risk probability is
Fake and unfounded request detection	Precision at K, false positive rate on first time reporters	Quality of the highest risk flags, and fairness to citizens with no history yet
Urgency assignment	Critical/High recall, false negative rate on Critical requests	The main safety metrics, catching genuinely urgent cases and never missing one
Urgency assignment	Weighted Cohen's kappa	Agreement with expert labels, penalizing a severe misclassification more than a minor one
Request prioritization	Precision at K, NDCG at K, Kendall rank correlation	Whether the most important requests actually surface near the top of the queue
Escalation and citizen credibility scoring	Mean absolute error against a synthetic citizen's hidden true honesty rate	How accurate the trust score is, checked against ground truth only we can see in a synthetic test
Escalation and citizen credibility scoring	False positive rate on new citizens	Fairness, a brand new citizen should never be judged on one report
Resolution time and external factors	Mean absolute error, RMSE, R squared	Accuracy of the predicted resolution time against held out data
Crew deployment and route optimization	Total distance, optimality gap, runtime	Efficiency and quality of the route plan against a simple nearest-crew baseline
NLP intake bot	Intent and slot filling accuracy	Whether the bot correctly captures what the citizen actually said
Advanced predictive alerting	Precision and recall of predicted demand against requests actually filed	Whether the weather based forecast genuinely matches what citizens report
Profanity filter	Precision, recall, false positive rate	Whether abusive language is caught without wrongly flagging clean text

Business Metrics

Metric	What It Shows
Percentage reduction in backlog	Whether recurring problems get solved for good instead of cycling through repeated temporary fixes
Average days to a permanent fix after escalation	Whether escalation actually speeds up a lasting resolution
Percentage reduction in total crew travel distance	The efficiency gained from optimized routing compared with a first in, first out baseline
Percentage of Critical and High requests acknowledged within target time	How responsive we are to genuinely urgent cases
Wasted crew trips avoided per month	The direct saving from catching fake or unfounded reports before a crew is sent
Percentage of genuine requests wrongly de-prioritized	The harm guardrail, tracked alongside the saving above so one is never improved at the other's expense
Citizens informed per resolved job	The value of grouping duplicates, one fix can now notify everyone who reported it
Context enrichment coverage	The percentage of requests where nearby hospital or school context was successfully attached
Number of citizens recognized on the leaderboard	How much the new recognition feature, medals, certificates, or subsidy, is actually being used
Percentage of predicted demand that matches requests actually filed	The real world value of the weather based alerting, not just a technical score

8. Implementation, Results, Challenges, Future Work, and Conclusion

These sections are optional at the proposal stage under the module guidelines and become mandatory in the final report, due 25 October 2026. We will document implementation progress, real experiment results against the metrics above, honest challenges encountered, and future work as the build proceeds, using the architecture and component ownership fixed in this proposal as the baseline plan against which we report our own progress and any deviations.

9. Supplementary Materials

Note on the Use of AI

This proposal, the system architecture diagram, and supporting project documents were drafted with AI assistance for formatting purposes.

Every technical decision, component ownership assignment, and scope commitment reflected here originates from the team's own component documentation and team discussions, the AI assistance was used for drafting, formatting, and synthesizing that material into this document, and for producing the architecture diagram from the team's specifications. The team has reviewed and takes ownership of the content.

Acknowledgements

The team thanks the course instructors for the practice module guidance that shaped this proposal's structure.

Appendix B Functions mapped to MR, RS and CGS

Function	Machine Reasoning (MR)	Reasoning Systems (RS)	Cognitive Systems (CGS)
Intake bot	N/A	Supervised classification, abstaining when unsure	Intent detection, slot tagging, sentence embeddings, dialogue
Emergency check	Weighted rules, fail closed	Threshold chosen on a recall first sweep	Lexicon based language screening
Agency and skill routing	Catalogue knowledge base, business rules	Learned baselines as evidence for the rules	N/A
Duplicate detection	Matching rules over evidence	Clustering (DBSCAN, HDBSCAN), grid search tuning	Fuzzy address matching
External factors	Spatial rules, knowledge representation of facilities	Threshold sensitivity analysis	N/A
Credibility and escalation	Escalation rule engine	Beta distribution model, reasoning under uncertainty	N/A
Fake request detection	Guardrail rules	Calibrated logistic regression, gradient boosting compared	N/A
Urgency	Forward chaining over hazard knowledge bases	Four learned model families compared	N/A
Prioritisation	Weighted multi criteria decision rules, guardrails	Ranking evaluation	N/A
Crew deployment and routing	Genetic algorithm, A* informed search, constraints	Benchmarks against greedy baselines	N/A
Predictive alerting	N/A	Demand forecasting with weather features	N/A
Admin questions	Read only query rules, refusal of write requests	TF IDF intent classifier	Natural language questions over data

Table 25. System functions against the course knowledge and techniques

Appendix C Installation and user guide

This appendix holds the two guides written for readers new to these tools.

The **Setup Guide** installs and starts the system on any computer with Docker, and lists the usual errors and their fixes.

The **Usage Handbook** explains what each person can do, screen by screen. To try it without installing anything, open http://157.230.191.175/ on a laptop or desktop (on a phone, see section 3.6). It may be taken down by January to February 2027, as it costs compute. Both guides are also in the repository's ProjectReport folder.

Setup Guide

Civic Operations · Team MANTIS · from nothing to a running app in about 30 minutes

This guide is for anyone, including people who have never used a terminal. You install one free program, download the project, type five commands, and open the app in your browser. Nothing is sent anywhere: the app runs only on your computer.

What you need

Item	Why
A Windows 10 or 11, macOS or Linux computer with 8 GB of memory or more	The app uses about 1 GB while running, Docker needs some too
About 10 GB of free disk space	The app, its models and the city road map
An internet connection for the first setup	To download Docker and the parts of the app. After that it runs offline
Docker Desktop, free	It runs the whole app in two sealed boxes called containers, the same on every computer

Step 1: install Docker Desktop

1. Go to docker.com, choose **Products**, then **Docker Desktop**, and download it for your computer.
2. Run the installer and accept the defaults. On Windows, leave **Use WSL 2** ticked. Restart when asked.
3. Open Docker Desktop and wait until the bottom left corner says **Engine running**. Keep it open while you use the app.

Step 2: get the project

Pick one of the two ways.

- **Download a ZIP (easiest).** Open https://github.com/DevelopingGod/IRS-PM-2026-08-17-AIS08FT-GRP-MANTIS-CivicOperations , click the green **Code** button, then **Download ZIP**. Unzip it, for example to your Documents folder.
- **With Git.** If you have Git installed, type: `git clone https://github.com/DevelopingGod/IRS-PM-2026-08-17-AIS08FT-GRP-MANTIS-CivicOperations.git`

Step 3: open a terminal in the right folder

- **Windows.** Open the project folder, then the **SystemCode** folder inside it. Click the address bar at the top, type `powershell` and press Enter. A blue window opens in that folder.
- **macOS.** Open **Terminal**, type `cd` (with a space), drag the **SystemCode** folder into the window, and press Enter.

You are in the right place when typing `dir` (Windows) or `ls` (macOS) lists a file called `docker-compose.yml`.

Step 4: start the app

Type each line and press Enter. Wait for each to finish before typing the next.

```
docker compose up --build -d
docker compose exec backend python -m app.backend.manage import-backlog --count 150
docker compose exec backend python -m app.backend.manage team-accounts > credentials.txt
```

1. The first line builds the app. **The first time takes 10 to 20 minutes.** Later starts take seconds.
2. The second line loads 150 real New York requests so the queue is not empty. It takes about a minute.
3. The third line creates sign in IDs and PINs for city staff, crews and the tech team, and saves them in a file called `credentials.txt` in the SystemCode folder. Open it with Notepad or TextEdit. Keep it private.

Step 5: open it

Open **http://localhost:8080** in Chrome, Edge, Firefox or Safari. Residents register with any email on the home page. City staff, crews and the tech team use the footer link **City staff, crew and tech team sign in** with an ID and PIN from `credentials.txt`. The Usage Handbook explains each screen.

Stopping, starting and starting fresh

To	Type in the SystemCode folder
Stop the app (your data is kept)	`docker compose stop`
Start it again	`docker compose start`, or `docker compose up -d`
Empty the queue and load fresh requests	`docker compose exec backend python -m app.backend.manage reset --yes` then the import-backlog line from step 4
Give someone a new PIN	`docker compose exec backend python -m app.backend.manage reset-pin THEIR-ID`
Remove the app completely	`docker compose down -v` (this deletes its data)

If something goes wrong

What you see	What it means	What to do
`docker` is not recognised, or command not found	Docker Desktop is not installed, or the terminal was opened before installing it	Install Docker Desktop, then close and reopen the terminal
Cannot connect to the Docker daemon, or error during connect	Docker Desktop is not running	Open Docker Desktop and wait for Engine running, then try again
no configuration file provided, or docker-compose.yml not found	The terminal is in the wrong folder	Go back to step 3 and open the terminal inside SystemCode
port is already allocated, or address already in use (8080)	Another program is using port 8080	Close that program, or open docker-compose.yml, change "8080:80" to "8081:80" and use http://localhost:8081
The build stops with a download or network error	The connection dropped while downloading	Run `docker compose up --build -d` again. It continues where it stopped

no space left on device	The disk is full	Free at least 10 GB. In Docker Desktop, Troubleshoot, Clean up data, removes old builds
WSL 2 installation is incomplete (Windows)	Windows needs its Linux layer for Docker	Follow the link in the message, install the update, restart, open Docker Desktop again
The page says Cannot reach the server	The backend is still starting, it loads its models first	Wait one or two minutes and reload. `docker compose ps` shows backend as healthy when ready
The page does not open at all	The containers are not running	Type `docker compose up -d`, wait a minute, reload
The queue is empty	No requests were loaded	Run the import-backlog line from step 4
The ID and PIN do not match	A typing mistake, or an old credentials file	Check the ID letter by letter. Five wrong PINs lock an ID for 15 minutes. Use reset-pin for a new PIN
credentials.txt is empty or missing	The third line in step 4 was not run, or was run in another folder	Run it again inside SystemCode. It creates new accounts each time
Very slow on an older computer	Memory is short	Close other programs. In Docker Desktop, Settings, Resources, give it at least 4 GB

Usage Handbook

Civic Operations · Team MANTIS · what each person can do, screen by screen

Civic Operations helps a city handle non emergency problems, such as potholes, broken street lights or noise, from the moment a resident reports one until a crew fixes it. Twelve reasoning components make suggestions and explain them. People stay in charge. This handbook covers the four kinds of user. The app must be running first: see the Setup Guide.

Who	How to get in	Main screens
Resident	Home page, register with any email, keep the user ID shown	Report, Track
City staff	Footer link, staff ID and PIN	Queue, Crew routes, Admin questions, Busy day alerts
Crew	Footer link, crew ID and PIN	Today's stops
Tech team	Footer link, tech ID and PIN	Components, Measurements, Accounts, System
Anyone	Features in the top menu	What the product does

Residents

Register

On the home page click **Yes, report it**. Enter any email and click **Create my user ID**. You get an ID such as NYC-ABCD-1234. Write it down: next time you sign in with it. Only a scrambled fingerprint of your email is kept.

Report a problem

- **Pick it.** Choose the kind of problem from a few clear choices, then the place and the details.
- **Or just tell us.** Type it in your own words, for example: there is a deep pothole on Atlantic Avenue near 4th Avenue in Brooklyn. The bot reads the problem, the address and the time, and asks only for what is missing.

> **If someone may be in danger**, the app stops with a full screen message to call 911. Nothing is filed and your words are not kept. That is on purpose: 311 is not for emergencies.

Track it

After sending you see a reference such as CO-551A4B and a line of stations, like a subway map, that light up as your report moves on. Ask the status bot questions such as where is my request. If the problem is still there after it was closed, say so and it comes back with more attention.

City staff

The queue

Every open request, ranked. Each card says why it is where it is, for example high urgency, official deadline 48 h, near a school. Click the numbers at the top to filter. Zoom the map to a borough with the buttons above it: the real streets appear. Click a dot or a card to open the request.

A request

- **Overview.** Agency, status, the reporter's record (never their identity), and the map.
- **Why, step by step.** What each component decided, and why.
- **Notes.** Notes for other staff.

- **Your decisions.** Approve an escalation, or confirm that a nearby school or hospital is affected. These count in the ranking only once you approve them.
- **Crew.** See which crew has it, give it to another crew, or **Pull back** to take it off its crew.

Crew routes

Crew deployment plans every crew's day and sends each route to its crew automatically. Open a crew to see its route on a street map: the black square is its depot, the green dot is where it is now, and the numbers are its stops in order. **Pull back** removes a job from the crew. A pulled back job waits for a person and is never sent again by itself. If the tech team switches Crew deployment off, nothing is sent automatically and you assign crews from each request.

Admin questions and busy day alerts

Ask the queue a question in plain English, for example how many open requests are in Brooklyn. It only reads, it never changes anything. Busy day alerts show which kinds of report are expected to rise on a day, from the weather.

Crews

After signing in you see today's stops in order, on a street map that starts at your depot. Open a stop, then choose what you found: fixed, cleared before you arrived, nothing found, no access, or another agency's job. Confirm, and the stop is ticked, your position on the map moves there, and the resident sees the result.

Tech team

- **Components.** Every component's health, with one switch each. Switched off, a component gives its safe default and the app keeps working. Staff are told.
- **Measurements.** Run the switch off measurement: what each component changes in the queue.
- **Accounts.** Create staff, crew and tech accounts. A PIN is shown once. Give a new PIN or turn an account off.
- **System.** How each component was chosen: methods compared, runs, and what shipped.

Common questions and problems

Question or problem	Answer
I lost my resident user ID	Register again with the same email: the app says it is already registered but never shows the ID, for privacy. Register with another email to continue.
My report did not go through and I saw a STOP screen	The Emergency check thought someone may be in danger. If nobody is, click I am safe, go back, and describe it again without words such as collapsed or fire.
The bot asks again for the address	It could not find a New York address. Give a number and street, or two crossing streets, and the borough.
A crew sees no stops	No route has been sent to that crew yet, Crew deployment is switched off, or staff pulled the jobs back. Wait a minute and reload.
A request I expected is not in Crew routes	It may be waiting for review, held as a duplicate, or there is no crew with the skill and time left today. The bottom of Crew routes lists why.
Staff sign in says locked	Five wrong PINs lock an ID for 15 minutes. Ask the tech team for a new PIN.
The page shows Cannot reach the server	The app is still starting or was stopped. Wait a minute and reload, or see the Setup Guide.
Numbers and positions differ from a colleague's	They depend on the time of day and on what was filed since. That is expected.

Appendix D Log book

Work by date from the project start on 5 September, from the tracker and git history. 14 MS Teams meetings, each over an hour, are listed after it.

Date	Who	Kind	What was done	Evidence
08 Sep	Sankalp	Setup	GitHub repository set up: project structure, meeting logs and templates. The full dataset kept local, a documented slice committed	Repository (git, 8 Sep)
09 Sep	Nabeel	Design	Design of his two assigned components and the initial data flow diagram, 1.5 hours	Tracker (9 Sep)
09 Sep	All	Proposal	Proposal kickoff deck, and an architecture that commits to all four technique groups	Joint architecture discussion; deck authorship needs separate verification
11 Sep	Nabeel	Research	NYC 311 field inventory: complaint families, responsible agencies, the input fields on 311 request pages	GitHub (git, 11 Sep)
12 Sep	Mohammad	Research	Analysis of the NYC 311 workflow and the mapping from agency to crew skill, 3 hours	Tracker (12 Sep)
12 Sep	All	Proposal	Project proposal and final system architecture written for submission	All-member proposal review and agreement
16 Sep	Sankalp	Build	Post proposal build baseline: data pipeline over NYC 311 Open Data, locked component input and output contracts	Data pipeline, processed tables, contracts (git, 18 Sep)
18 Sep	Sankalp	Planning	Planning session: build plan, task board of 31 tasks with owners and due dates, decisions log	Tracker: Task Board T01 to T31, Decisions Log (tracker)
18 Sep	Sankalp	Research	Daily demand baselines for Predictive alerting, data preparation notebook rerun	Notebook 01 (git, 18 Sep)
19 Sep	Sankalp	Documentation	Read Me First guide, a personal reference guide for each member, shared system context for consistent work	05_Team_Workspace/01_Start_Here and 02_Personal_Guides (git, 19 Sep)
23 Sep	All	Meeting	Team meeting: views for admin, management, crew and citizen, how to show the IRS concepts, core components in the presentation	All-member discussion of system views and IRS concepts
25 Sep	Khalid	Build	First Urgency component upload	GitHub upload (git, 25 Sep)
26 Sep	All	Presentation	Animated demo, research gap table and technique, component and metric tree for the presentation	Presentation discussion; individual contributions to animation and diagrams need verification
27 Sep	All	Presentation	Speaker cue card and examiner question preparation	Preparation materials exist; individual participation is not established
28 Sep	All	Presentation	Proposal presentation to the examiners	Presentation deliverable and collective participation recorded in work log; attendance or speaking roles need confirmation
05 Oct	Mohammad	Research	Ran the language model experiment notebook on his GPU machine	Verified GPU run results (git, 5 Oct)
05 Oct	Sankalp	Research	Component specification, data findings document, 156 new hand written test lines, second experiment ipynb	05_Team_Workspace/03_Specification_and_Contracts (git, 5 Oct)
07 Oct	Nabeel	Build	Duplicate detection and Fake request detection v0.1.0: methods, test harness, moved onto the shared component layout	duplicate_detection/docs/CHANGELOG.md (7 Oct)
07 Oct	Sankalp	Build	Intake bot, Emergency check, Credibility and Predictive alerting built against the GPU results, evidence pack and experiment register	03_Research/evidence (git, 7 Oct)
08 Oct	Sankalp	Research	Credibility fairness gate closed, glossary, slot filling measured on hand marked spans, contribution report	Gates in component_status.json (git, 8 Oct)
09 Oct	Sankalp	Build	Shipped models committed, repository tidied	01_Application/models (git, 9 Oct)
11 Oct	Khalid	Build	Urgency component: rules, hazard knowledge base, service impact rules, tests	Branch khalid_code (git, 11 Oct)
12 Oct	Khalid	Build	Urgency rules revised, the single file version replaced by the component	Branch khalid_code (git, 12 Oct)
13 Oct	Khalid	Build	Urgency reorganised into its own folder: README, hazard knowledge base, service impact rules, tests. Working session with Sankalp	Branch khalid_code (git, 13 Oct), Meetings sheet
14 Oct	Mohammad	Research	Referral advisory rule for Agency and skill routing, learned from the training data	referral_knowledge.json, generated 14 Oct
14 Oct	Nabeel	Research	Duplicate pairs labelled by hand on the full real table, matcher tuned, HDBSCAN fix	duplicate_detection/docs/CHANGELOG.md (14 Oct)
15 Oct	Khalid	Build	Prioritisation ready for team review: official SLA lookup, starvation protection, ageing, documentation	C11_Request_Prioritization/README.md, status dated 15 Oct

15 Oct	Mohammad	Delivery	Agency and skill routing v1.1.0 with its research report and tests. External factors: 4,445 facilities acquired from OpenStreetMap	Delivered package, report built 15 Oct. external_factors/data/osm/acquisition_report.json (15 Oct)
15 Oct	Nabeel	Build	Duplicate detection and Fake request detection, tuned on 295 hand checked real pairs	Branch nabeel_c05_c06 (git, 15 Oct)
15 Oct	Nabeel	Research	Admin questions v1.1.0 benchmarked on 74 gold questions written before any strategy ran, results and change logs	Branch nabeel_c12 (git, 15 Oct)
15 Oct	Sankalp	Build	App backend with every component as a measured plugin, citizen, crew and staff portals	01_Application/app (git, 15 Oct)
16 Oct	Khalid	Delivery	Urgency and Prioritisation final version on his branch	Commit 906b77a, pinned in the app (git, 16 Oct)
16 Oct	Mohammad	Build	NYC road network built for crew routing	crew_deployment_and_routing roads_manifest.json, built 16 Oct
16 Oct	Sankalp	Review	Reviewed Urgency and Prioritisation on a fresh copy, sent findings to Khalid	Review message (16 to 18 Oct)

Table 26. Log book: work by date

Date	Mode	Topic	Hours	Who
29 Aug	In person	Kickoff: brainstorming and choosing the project idea	2	All four members
05 Sep	MS Teams	First meeting: system components, evaluation points, dataset, GitHub	2	All four members
07 Sep	MS Teams	Second meeting: report on assigned components, input and output template	1.5	All four members
09 Sep	MS Teams	Component design and the initial data flow diagram	1.5	All four members
11 Sep	MS Teams	Project proposal content agreed	1.5	All four members
12 Sep	MS Teams	Proposal finalised for submission	over 1	All four members
19 Sep	MS Teams	Team meeting	over 1	All four members
23 Sep	MS Teams	Portals and presentation planning	over 1	All four members
25 Sep	MS Teams	Team meeting	over 1	All four members
26 Sep	MS Teams	Team meeting	over 1	All four members
02 Oct	MS Teams	Team meeting	over 1	All four members
05 Oct	MS Teams	Team meeting	over 1	All four members
09 Oct	MS Teams	Team meeting	over 1	All four members
13 Oct	MS Teams	Working session	over 1	Sankalp and Khalid
20 Oct	MS Teams	Team meeting	over 1	All four members

Table 27. Log book: team meetings